

A collection of electronic components including an Arduino Mega 2560, resistors, LEDs, a breadboard, jumper wires, a USB cable, and a potentiometer.

Introduction

Hardware required

Arduino board *1

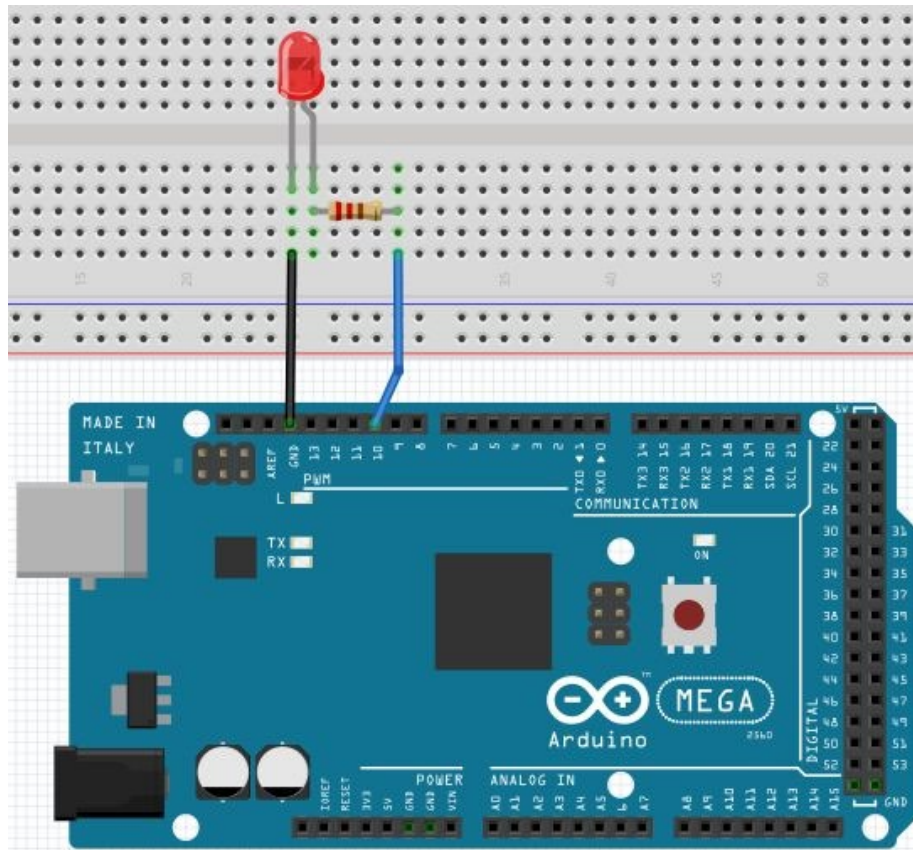
USB cable *1

Sample program

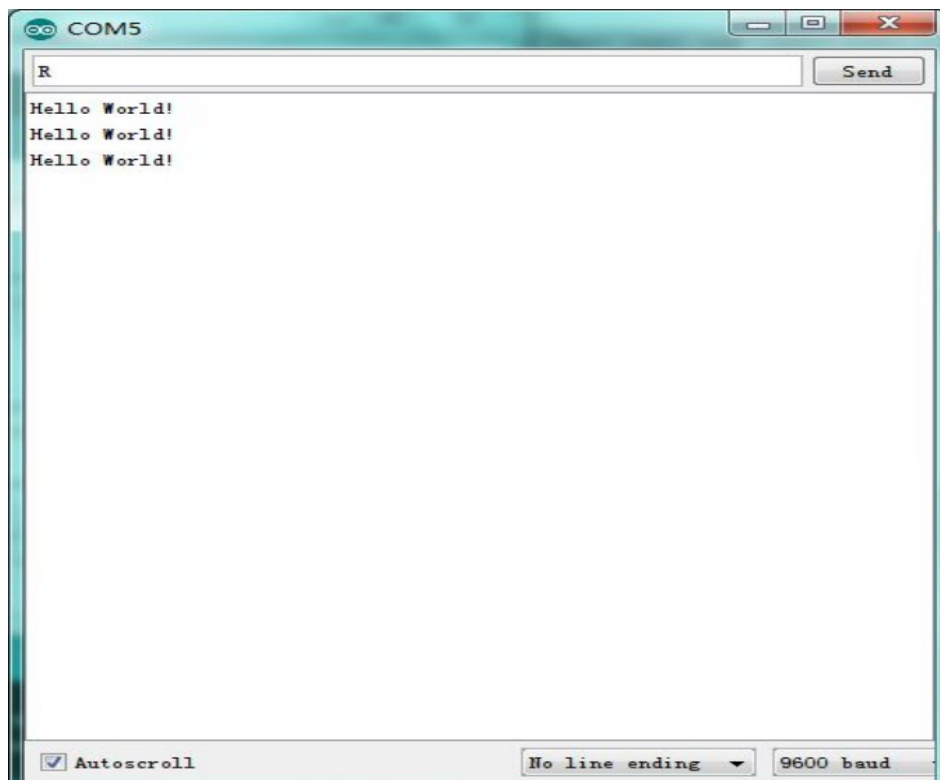
After installing driver for Arduino, let's open Arduino software and compile code that enables Arduino to print "Hello World!" under your instruction. Of course, you can compile code for Arduino to continuously echo "Hello World!" without instruction. A simple If () statement will do the instruction trick. With the onboard LED connected to pin 13, we can instruct the LED to blink first when Arduino gets an instruction and then print "Hello World!".

```
////////////////////////////////////
int val;//define variable val
int ledpin=13;// define digital interface 13
void setup()
{
  Serial.begin(9600);// set the baud rate at 9600 to match the software set up. When connected to
  a specific device, (e.g. bluetooth), the baud rate needs to be the same with it.
  pinMode(ledpin,OUTPUT);// initialize digital pin 13 as output. When using I/O ports on an
  Arduino, this kind of set up is always needed.
}
void loop()
{
  val=Serial.read();// read the instruction or character from PC to Arduino, and assign them to
  Val.
  if(val=='R')// determine if the instruction or character received is "R".
  { // if it's "R",
    digitalWrite(ledpin,HIGH);// set the LED on digital pin 13 on.
    delay(500);
    digitalWrite(ledpin,LOW);// set the LED on digital pin 13 off.
    delay(500);
    Serial.println("Hello World!");// display"Hello World! "string.
  }
}
////////////////////////////////////
```

Result

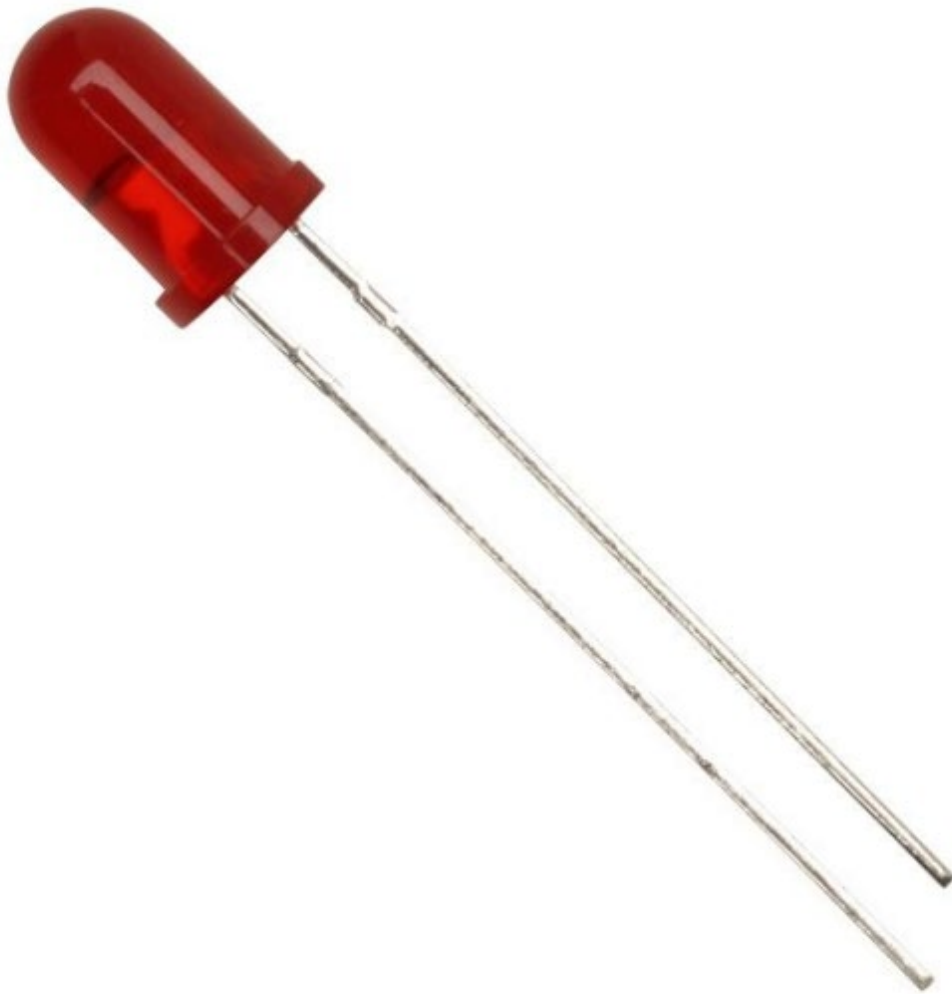


Click serial port monitor, Input R, LED 13 will blink once, PC will receive information from Arduino: Hello World



After you choose the right port, the experiment should be easy for you!

Project 2: LED blinking



Introduction

Blinking LED experiment is quite simple. In the "Hello World!" program, we have come across LED. This time, we are going to connect an LED to one of the digital pins rather than using LED13, which is soldered to the board. Except an Arduino and an USB cable, we will need extra parts as below:

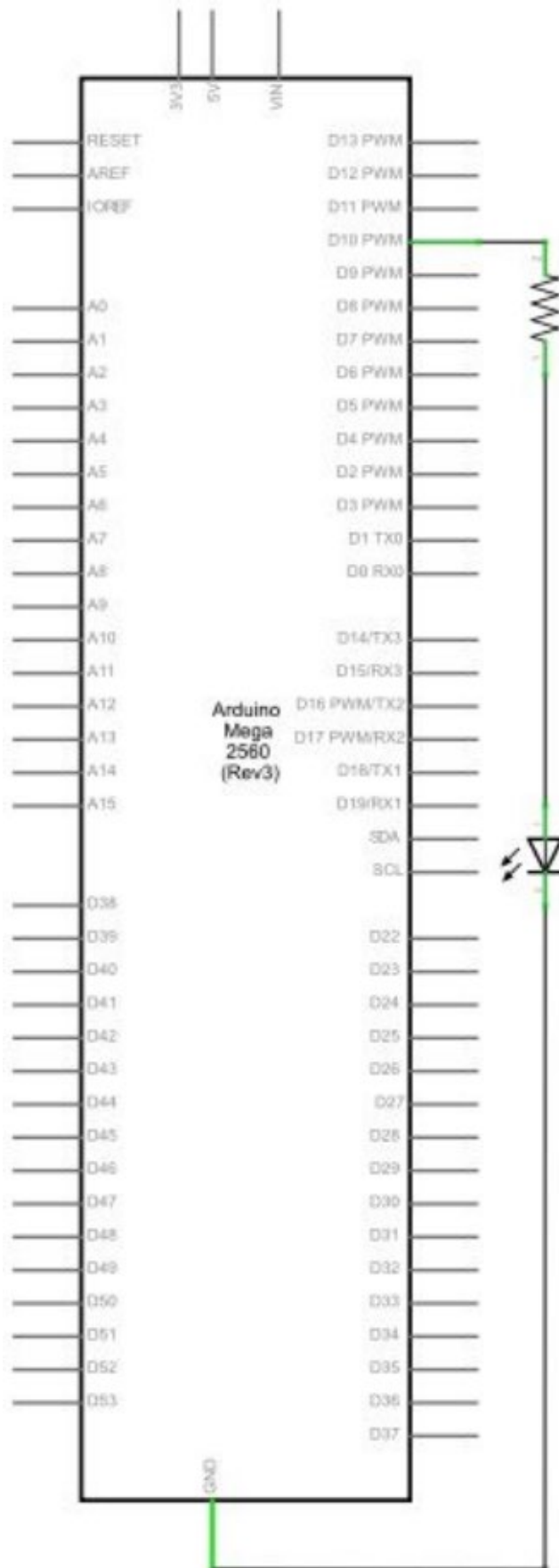
Hardware required

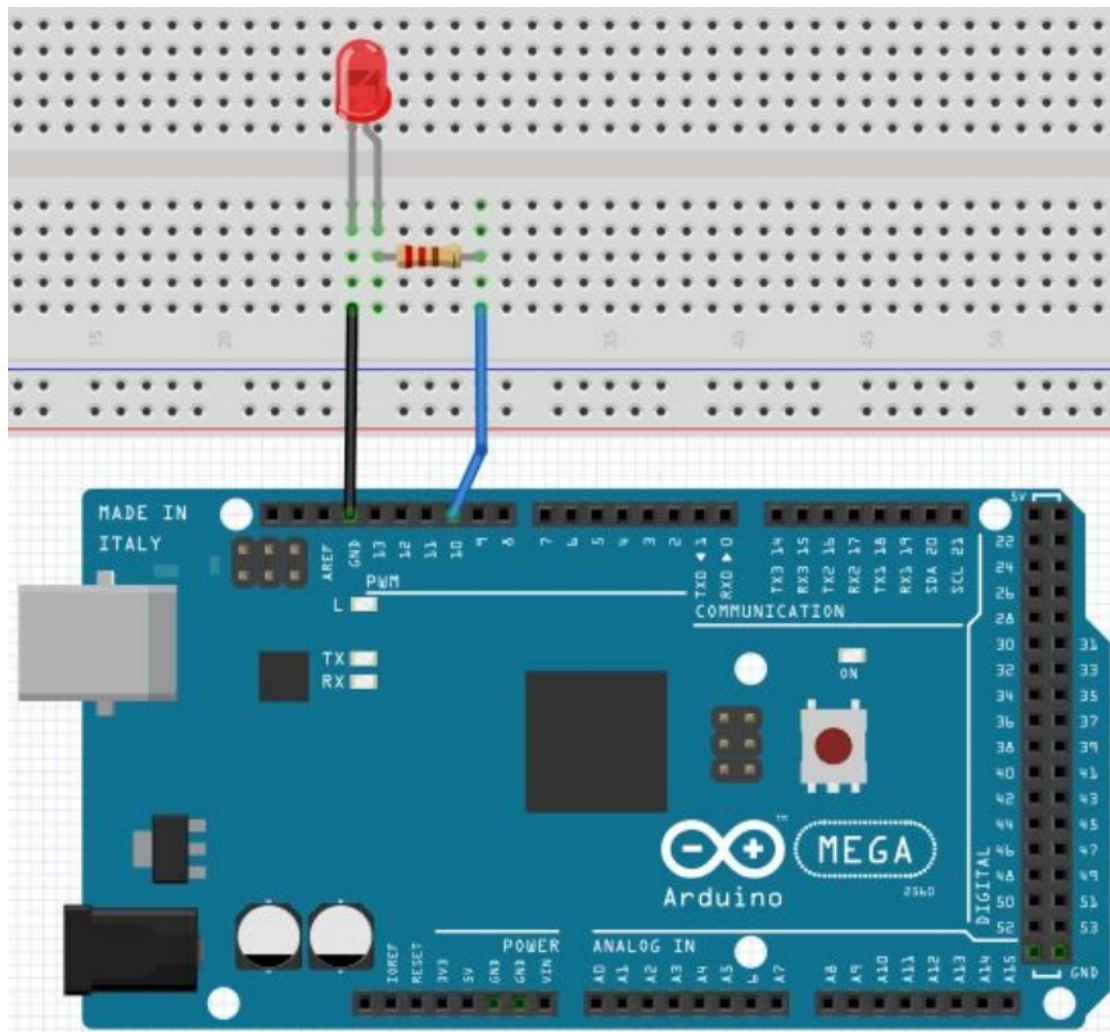
Red M5 LED*1
220 Ω resistor*1
Breadboard*1
Breadboard jumper wires

Circuit connection

We follow below diagram from the experimental schematic link. Here we use digital pin 10. We

connect LED to a 220 ohm resistor to avoid high current damaging the LED





Sample program

```

////////////////////////////////////
int ledPin = 10; // define digital pin 10.
void setup()
{
  pinMode(ledPin, OUTPUT); // define pin with LED connected as output.
}
void loop()
{
  digitalWrite(ledPin, HIGH); // set the LED on.
  delay(1000); // wait for a second.
  digitalWrite(ledPin, LOW); // set the LED off.
  delay(1000); // wait for a second
}
////////////////////////////////////

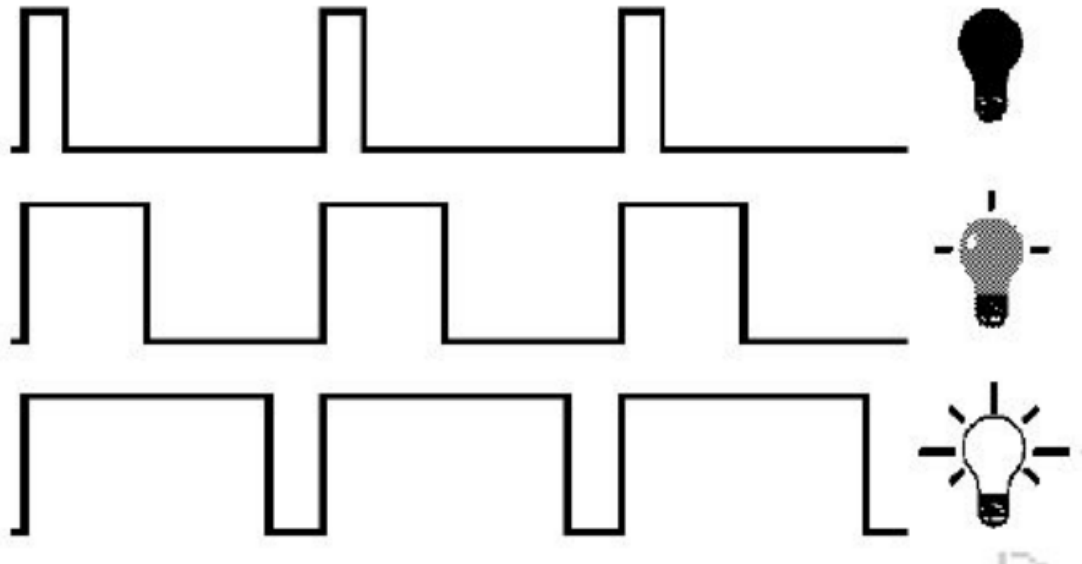
```

Result

After downloading this program, in the experiment, you will see the LED connected to pin 10

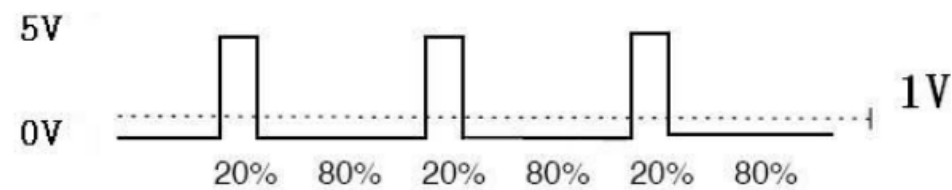
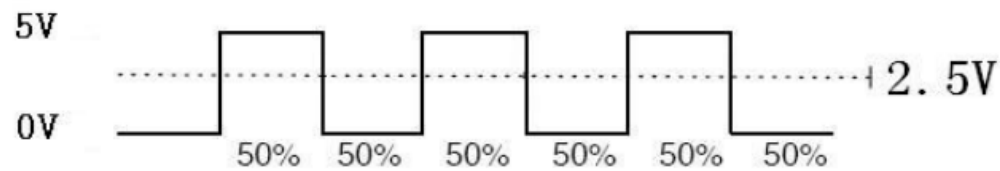
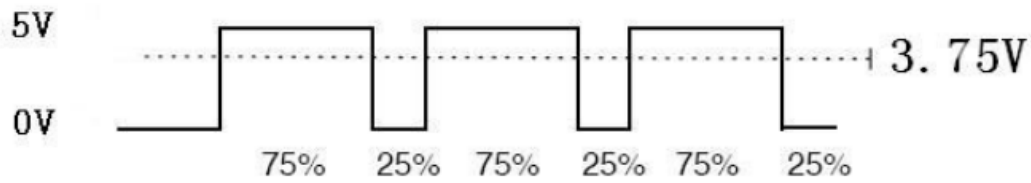
turning on and off, with an interval approximately one second.
The blinking LED experiment is now completed. Thank you!

Project 3: PWM light control

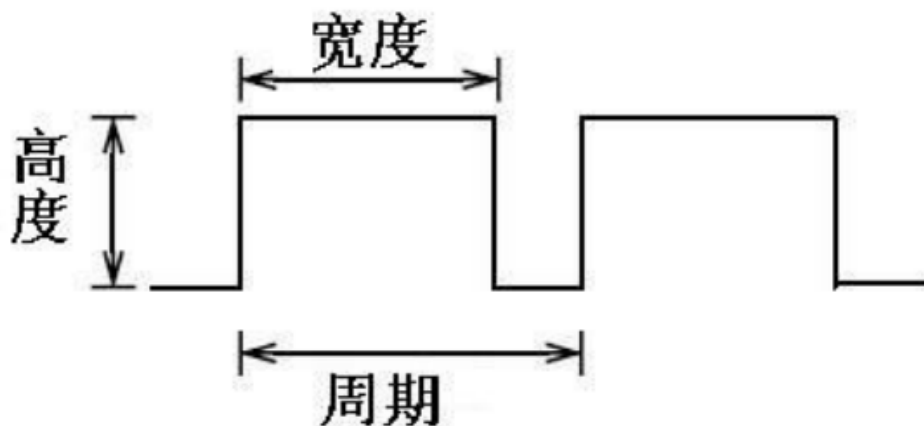


Introduction

PWM, short for Pulse Width Modulation, is a technique used to encode analog signal level into digital ones. A computer cannot output analog voltage but only digital voltage values such as 0V or 5V. So we use a high resolution counter to encode a specific analog signal level by modulating the duty cycle of PMW. The PWM signal is also digitalized because in any given moment, fully on DC power supply is either 5V (ON), or 0V (OFF). The voltage or current is fed to the analog load (the device that uses the power) by repeated pulse sequence being ON or OFF. Being on, the current is fed to the load; being off, it's not. With adequate bandwidth, any analog value can be encoded using PWM. The output voltage value is calculated via the on and off time. Output voltage = (turn on time/pulse time) * maximum voltage value



PWM has many applications: lamp brightness regulating, motor speed regulating, sound making, etc.
The following are the three basic parameters of PMW:



1. The amplitude of pulse width (minimum / maximum)
2. The pulse period (The reciprocal of pulse frequency in 1 second)
3. The voltage level (such as: 0V-5V)

There are 6 PWM interfaces on Arduino, namely digital pin 3, 5, 6, 9, 10, and 11. In previous experiments, we have done "button-controlled LED", using digital signal to control digital pin, also one about potentiometer. This time, we will use a potentiometer to control the brightness of the LED.

Hardware required

Variable resistor*1
Red M5 LED*1
220Ω resistor*1
Breadboard*1

Breadboard jumper wires

Circuit connection

Width

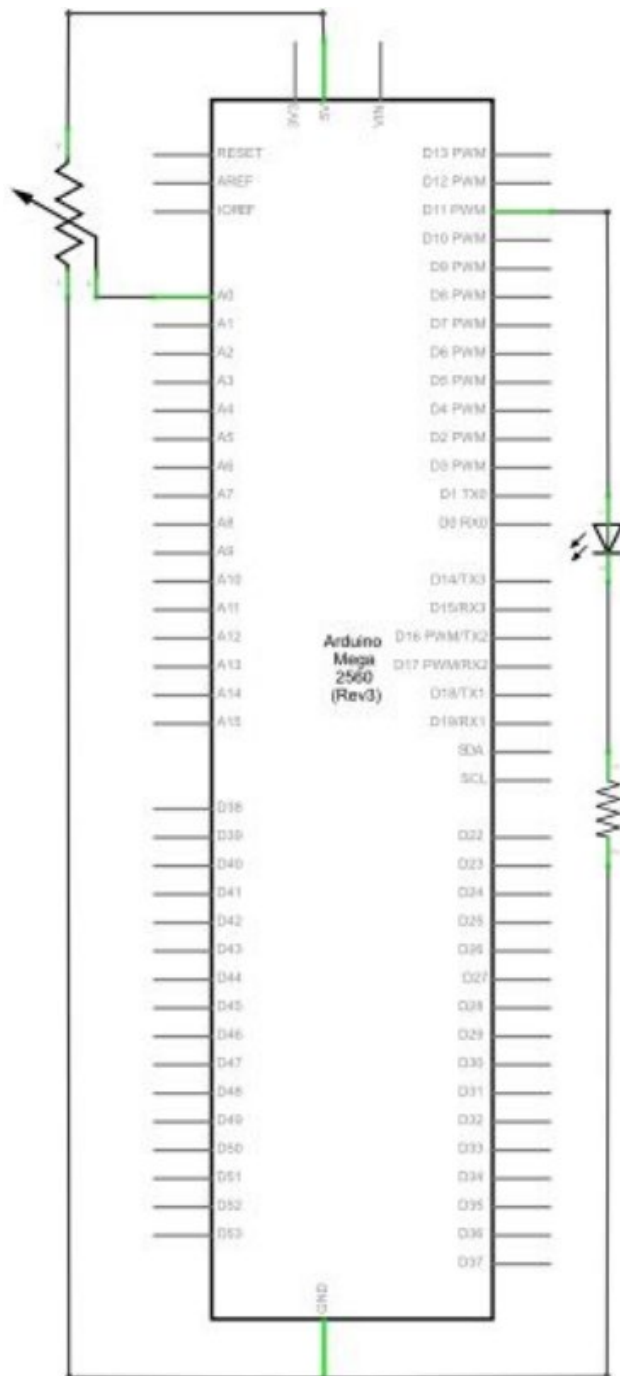
Cycle

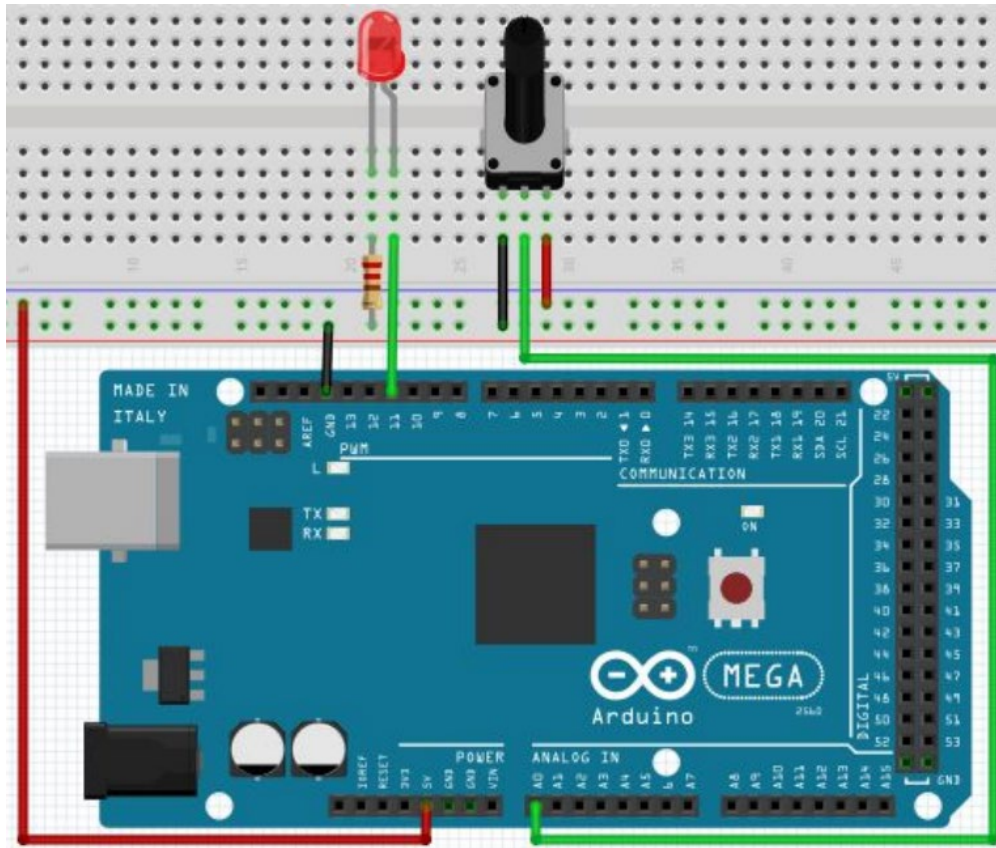
Level

10

The input of potentiometer is analog, so we connect it to analog port, and LED to PWM port.

Different PWM signal can regulate the brightness of the LED





Sample program

In the program compiling process, we will use the analogWrite (PWM interface, analog value) function. In this experiment, we will read the analog value of the potentiometer and assign the value to PWM port, so there will be corresponding change to the brightness of the LED. One final part will be displaying the analog value on the screen. You can consider this as the "analog value reading" project adding the PWM analog value assigning part. Below is a sample program for your reference.

```

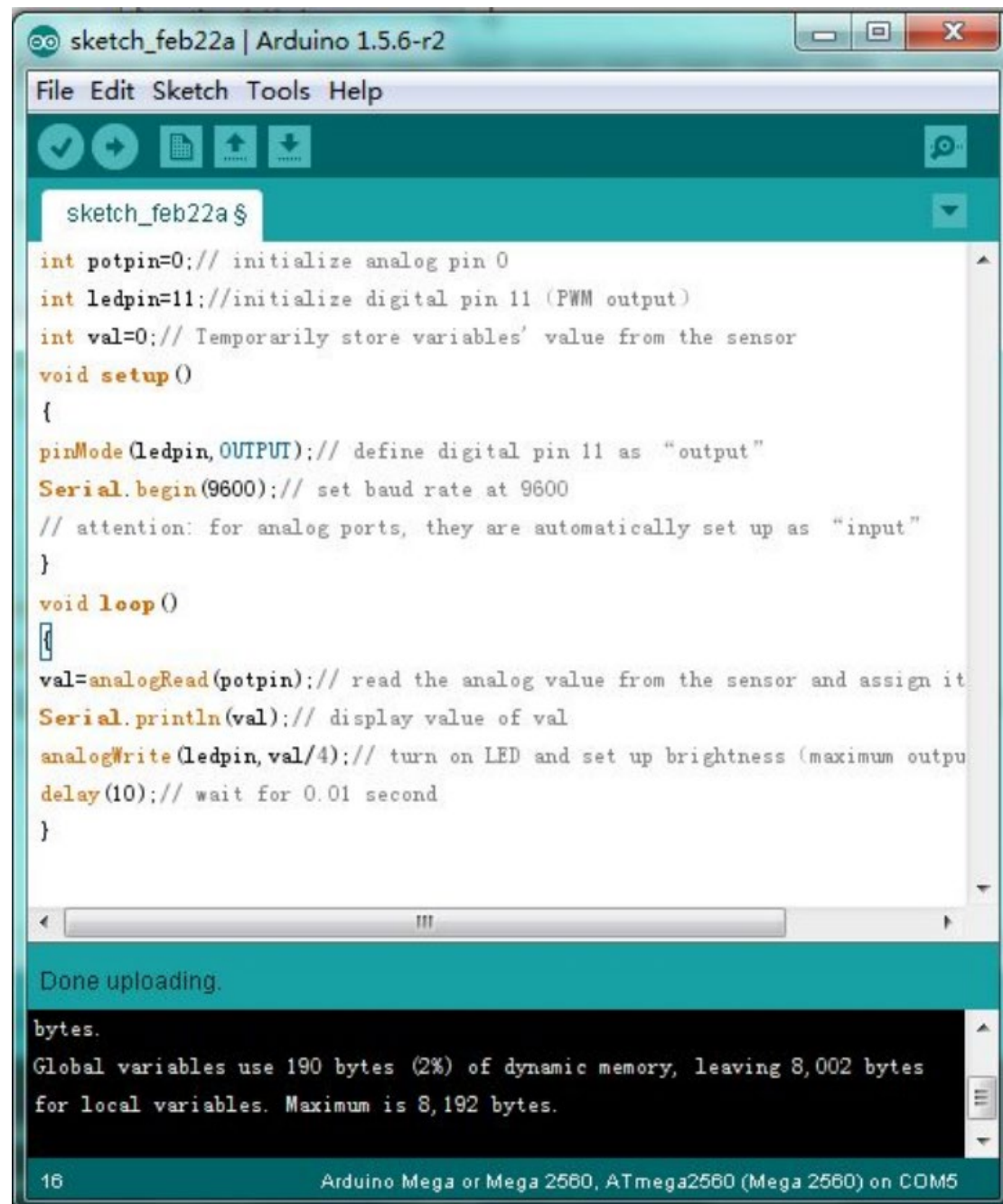
////////////////////////////////////
int potpin=0;// initialize analog pin 0
int ledpin=11;//initialize digital pin 11 (PWM output)
int val=0;// Temporarily store variables' value from the sensor
void setup()
{
  pinMode(ledpin,OUTPUT);// define digital pin 11 as "output"
  Serial.begin(9600);// set baud rate at 9600
  // attention: for analog ports, they are automatically set up as "input"
}
void loop()
{
  val=analogRead(potpin);// read the analog value from the sensor and assign it to val
  Serial.println(val);// display value of val
  analogWrite(ledpin,val/4);// turn on LED and set up brightness (maximum output of PWM is 255)
}

```

```

delay(10); // wait for 0.01 second
}
//////////

```



Result

After downloading the program, when we rotate the potentiometer knob, we can see changes of the displaying value, also obvious change of the LED brightness on the breadboard.

Project 4: Traffic light



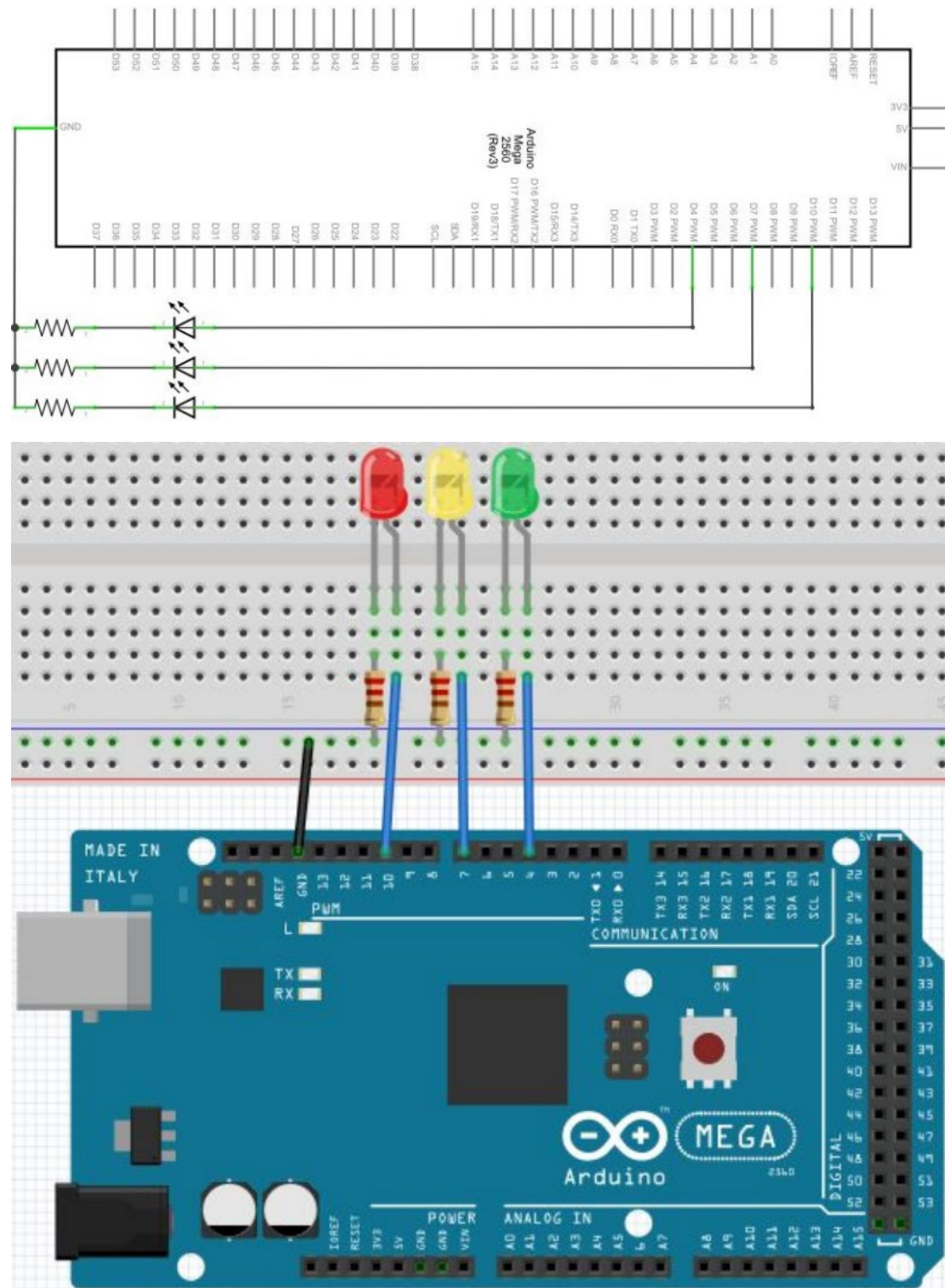
Introduction

In the previous program, we have done the LED blinking experiment with one LED. Now, it's time to up the stakes and do a bit more complicated experiment-traffic lights. Actually, these two experiments are similar. While in this traffic lights experiment, we use 3 LEDs with different color other than 1 LED.

Hardware required

- Arduino board *1
- USB cable *1
- Red M5 LED*1
- Yellow M5 LED*1
- Green M5 LED*1
- 220 Ω resistor *3
- Breadboard*1
- Breadboard jumper wires

Circuit connection



Sample program

Since it is a simulation of traffic lights, the blinking time of each LED should be the same with those in traffic lights system. In this program, we use Arduino delay () function to control delay time, which is much simpler than C language.

```

////////////////////////////////////
int redled =10; // initialize digital pin 8.
int yellowled =7; // initialize digital pin 7.
int greenled =4; // initialize digital pin 4.
void setup()
{
pinMode(redled, OUTPUT); // set the pin with red LED as “output”
pinMode(yellowled, OUTPUT); // set the pin with yellow LED as “output”
pinMode(greenled, OUTPUT); // set the pin with green LED as “output”
}
void loop()
{
digitalWrite(greenled, HIGH); // turn on green LED
delay(5000); // wait 5 seconds
digitalWrite(greenled, LOW); // turn off green LED
for(int i=0;i<3;i++) // blinks for 3 times
{
delay(500); // wait 0.5 second
digitalWrite(yellowled, HIGH); // turn on yellow LED
delay(500); // wait 0.5 second
digitalWrite(yellowled, LOW); // turn off yellow LED
}
delay(500); // wait 0.5 second
digitalWrite(redled, HIGH); // turn on red LED
delay(5000); // wait 5 second
digitalWrite(redled, LOW); // turn off red LED
}
////////////////////////////////////

```

Result

When the uploading process is completed, we can see traffic lights of our own design.

Note: this circuit design is very similar with the one in LED chase effect.

The green light will be on for 5 seconds, and then off., followed by the yellow light blinking for 3 times, and then the red light on for 5 seconds, forming a cycle. Cycle then repeats.

Experiment is now completed, thank you.

Project 5: LED chasing effect



Introduction

We often see billboards composed of colorful LEDs. They are constantly changing to form various effects. In this experiment, we compile a program to simulate chase effect.

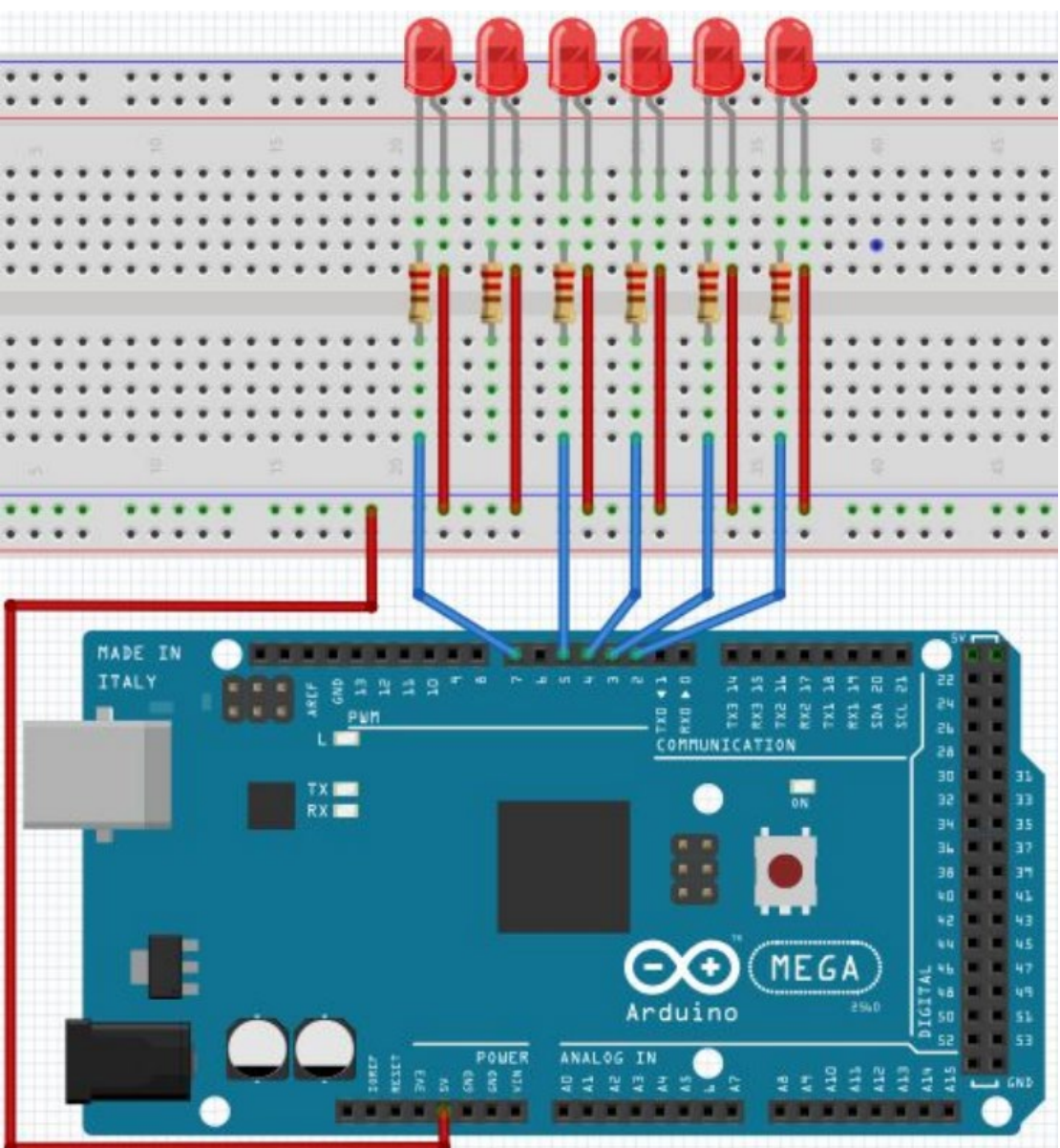
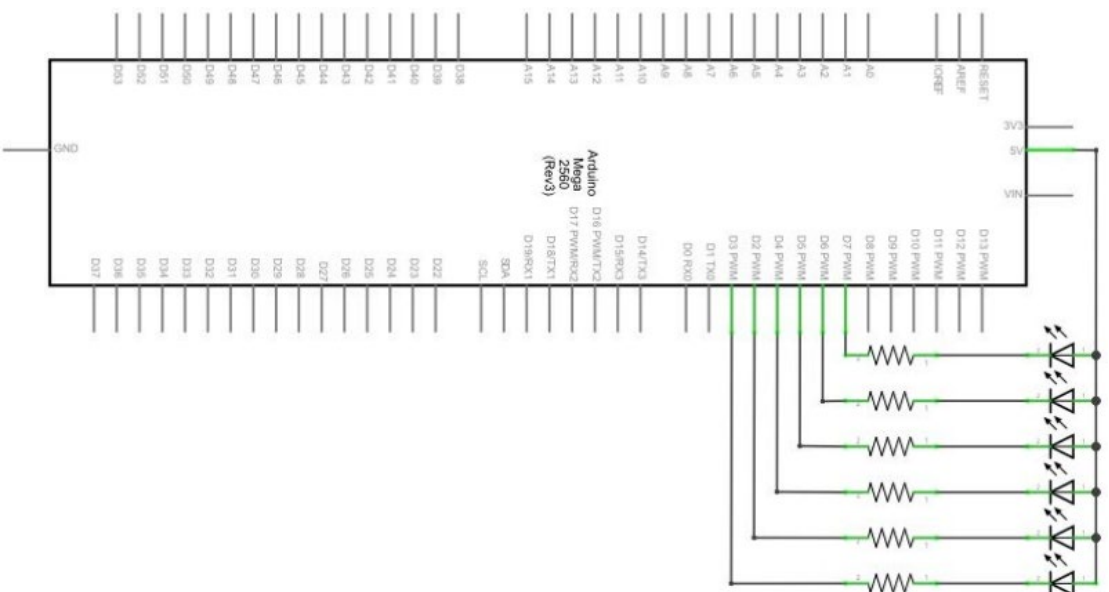
Hardware required

Led *6

220 Ω resistor *6

Breadboard jumper wires

Circuit connection



Sample program

```
////////////////////////////////////
int BASE = 2 ; // the I/O pin for the first LED
int NUM = 6;
// number of LEDs
void setup()
{
  for (int i = BASE; i < BASE + NUM; i++)
  {
    pinMode(i, OUTPUT);
    // set I/O pins as output
  }
}
void loop()
{
  for (int i = BASE; i < BASE + NUM; i++)
  {
    digitalWrite(i, LOW);
    // set I/O pins as “low”, turn on LEDs one by one.
    delay(200);
    // delay
  }
  for (int i = BASE; i < BASE + NUM; i++)
  {
    digitalWrite(i, HIGH);
    // set I/O pins as “high”, turn off LEDs one by one
    delay(200);
    // delay
  }
}
////////////////////////////////////
```

Result

You can see the LEDs blink by sequence.

Project 6: Button-controlled LED



Introduction

I/O port means interface for INPUT and OUTPUT. Up until now, we have only used its OUTPUT function. In this experiment, we will try to use the input function, which is to read the output value of device connecting to it. We use 1 button and 1 LED using both input and output to give you a better understanding of the I/O function. Button switches, familiar to most of us, are a switch value (digital value) component. When it's pressed, the circuit is in closed (conducting) state.

Hardware required

Button switch*1

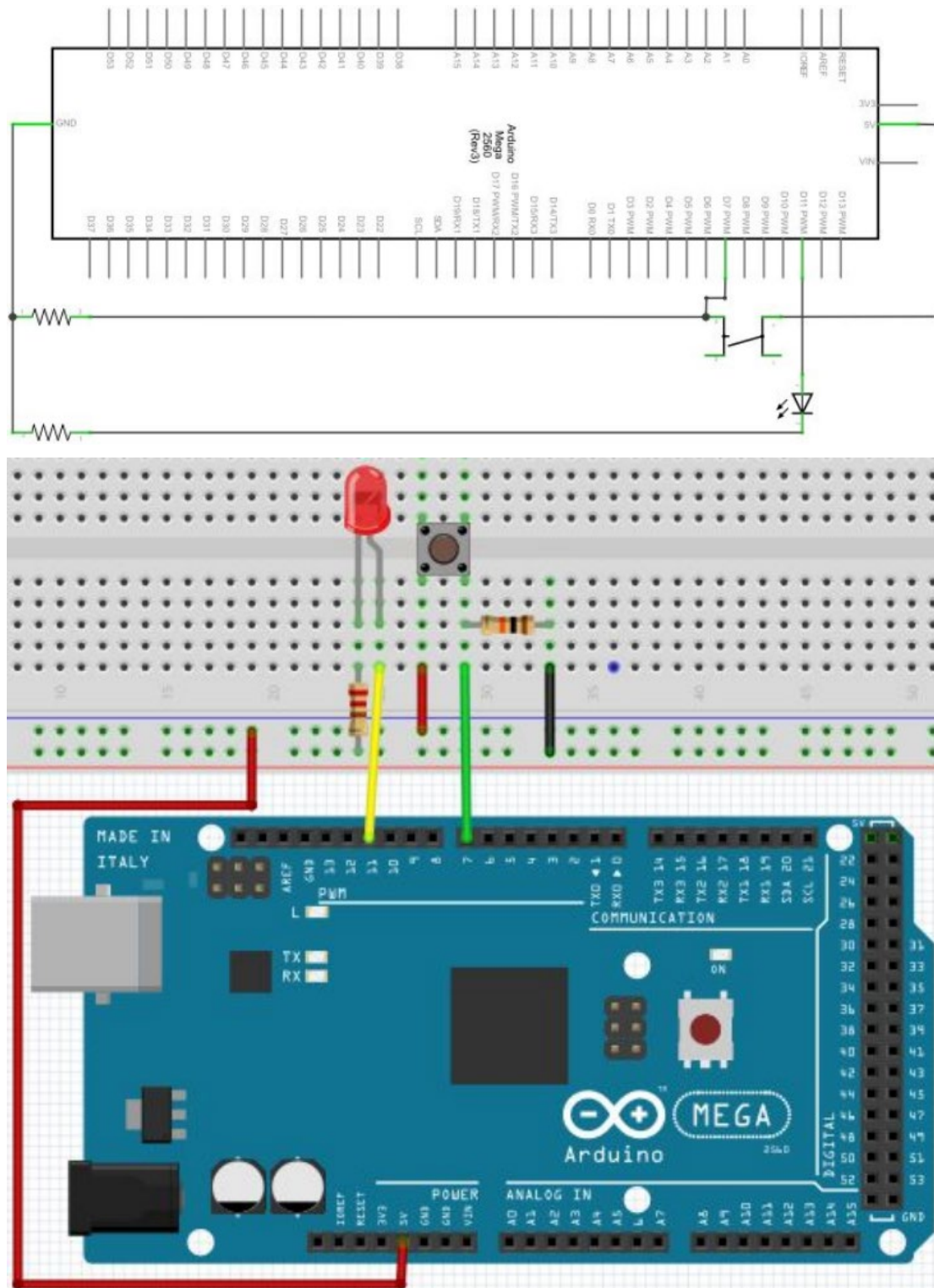
Red M5 LED*1

220 Ω resistor*1

10K Ω resistor*1

Breadboard*1

Breadboard jumper wires



Sample program

Now, let's begin the compiling. When the button is pressed, the LED will be on. After the previous study, the coding should be easy for you. In this program, we add a statement of judgment. Here, we use an `if ()` statement.

Arduino IDE is based on C language, so statements of C language such as while, switch etc. can certainly be used for Arduino program.

When we press the button, pin 7 will output high level. We can program pin 11 to output high level and turn on the LED. When pin 7 outputs low level, pin 11 also outputs low level and the LED remains off.

```
////////////////////////////////////
int ledpin=11;// initialize pin 11
int inpin=7;// initialize pin 7
int val;// define val
void setup()
{
pinMode(ledpin,OUTPUT);// set LED pin as “output”
pinMode(inpin,INPUT);// set button pin as “input”
}
void loop()
{
val=digitalRead(inpin);// read the level value of pin 7 and assign it to val
if(val==LOW)// check if the button is pressed, if yes, turn on the LED
{ digitalWrite(ledpin,LOW);}
else
{ digitalWrite(ledpin,HIGH);}
}
////////////////////////////////////
```

Result

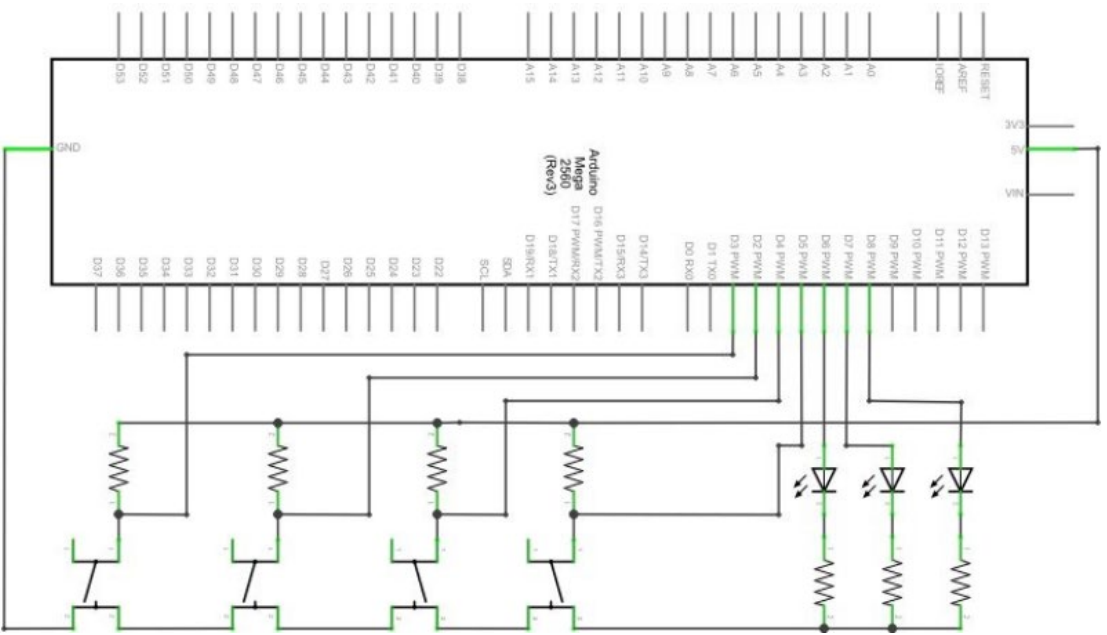
When the button is pressed, LED is on, otherwise, LED remains off. After the above process, the button controlled LED experiment is completed. The simple principle of this experiment is widely used in a variety of circuit and electric appliances. You can easily come across it in your every day life. One typical example is when you press a certain key of your phone, the backlight will be on.

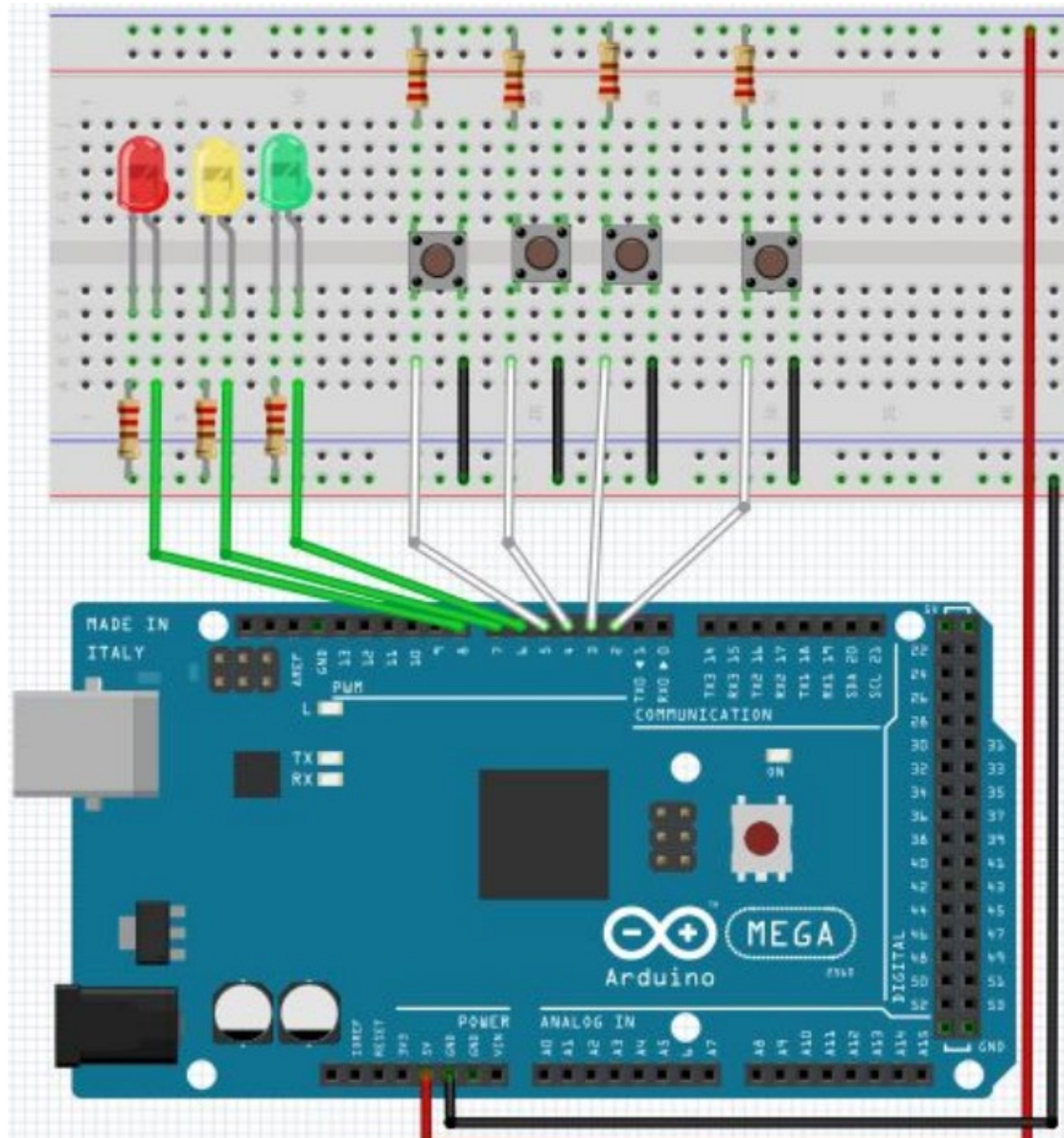
Project 7: Responder experiment

Introduction

After completing all the previous experiments, we believe you will find this one easy. In this program, we have 3 buttons and a reset button controlling the corresponding 3 LEDs, using 7 digital I/O pins.

Circuit connection





Sample program

```

////////////////////////////////////
int redled=8;
// set red LED as "output"
int yellowled=7; // set yellow LED as "output"
int greenled=6;
// set green LED as "output"
int redpin=5;
// initialize pin for red button
int yellowpin=4; // initialize pin for yellow button
int greenpin=3;
// initialize pin for green button
int restpin=2;
// initialize pin for reset button

```

```

int red;
int yellow;
int green;
void setup()
{
  pinMode(redled,OUTPUT);
  pinMode(yellowled,OUTPUT);
  pinMode(greenled,OUTPUT);
  pinMode(redpin,INPUT);
  pinMode(yellowpin,INPUT);
  pinMode(greenpin,INPUT);
}
void loop() // repeatedly read pins for buttons
{
  red=digitalRead(redpin);
  yellow=digitalRead(yellowpin);
  green=digitalRead(greenpin);
  if(red==LOW)RED_YES();
  if(yellow==LOW)YELLOW_YES();
  if(green==LOW)GREEN_YES();
}
void RED_YES()// execute the code until red light is on; end cycle when reset button is pressed
{
  while(digitalRead(restpin)==1)
  {
    digitalWrite(redled,HIGH);
    digitalWrite(greenled,LOW);
    digitalWrite(yellowled,LOW);
  }
  clear_led();
}
void YELLOW_YES()// execute the code until yellow light is on; end cycle when reset button is
pressed
{
  while(digitalRead(restpin)==1)
  {
    digitalWrite(redled,LOW);
    digitalWrite(greenled,LOW);
    digitalWrite(yellowled,HIGH);
  }
  clear_led();
}
void GREEN_YES()// execute the code until green light is on; end cycle when reset button is
pressed

```

```

{
while(digitalRead(restpin)==1)
{
digitalWrite(redled,LOW);
digitalWrite(greenled,HIGH);
digitalWrite(yellowled,LOW);
}
clear_led();
}
void clear_led()// all LED off
{
digitalWrite(redled,LOW);
digitalWrite(greenled,LOW);
digitalWrite(yellowled,LOW);
}
////////////////////////////////////

```

Result

Whichever button is pressed first, the corresponding LED will be on!

Then press the REST button to reset.

After the above process, we have built our own simple responder.

Project 8: Active buzzer



Introduction

Active buzzer is widely used on computer, printer, alarm, electronic toy, telephone, timer etc as a sound making element. It has an inner vibration source. Simply connect it with 5V power supply, it can buzz continuously.

Hardware required

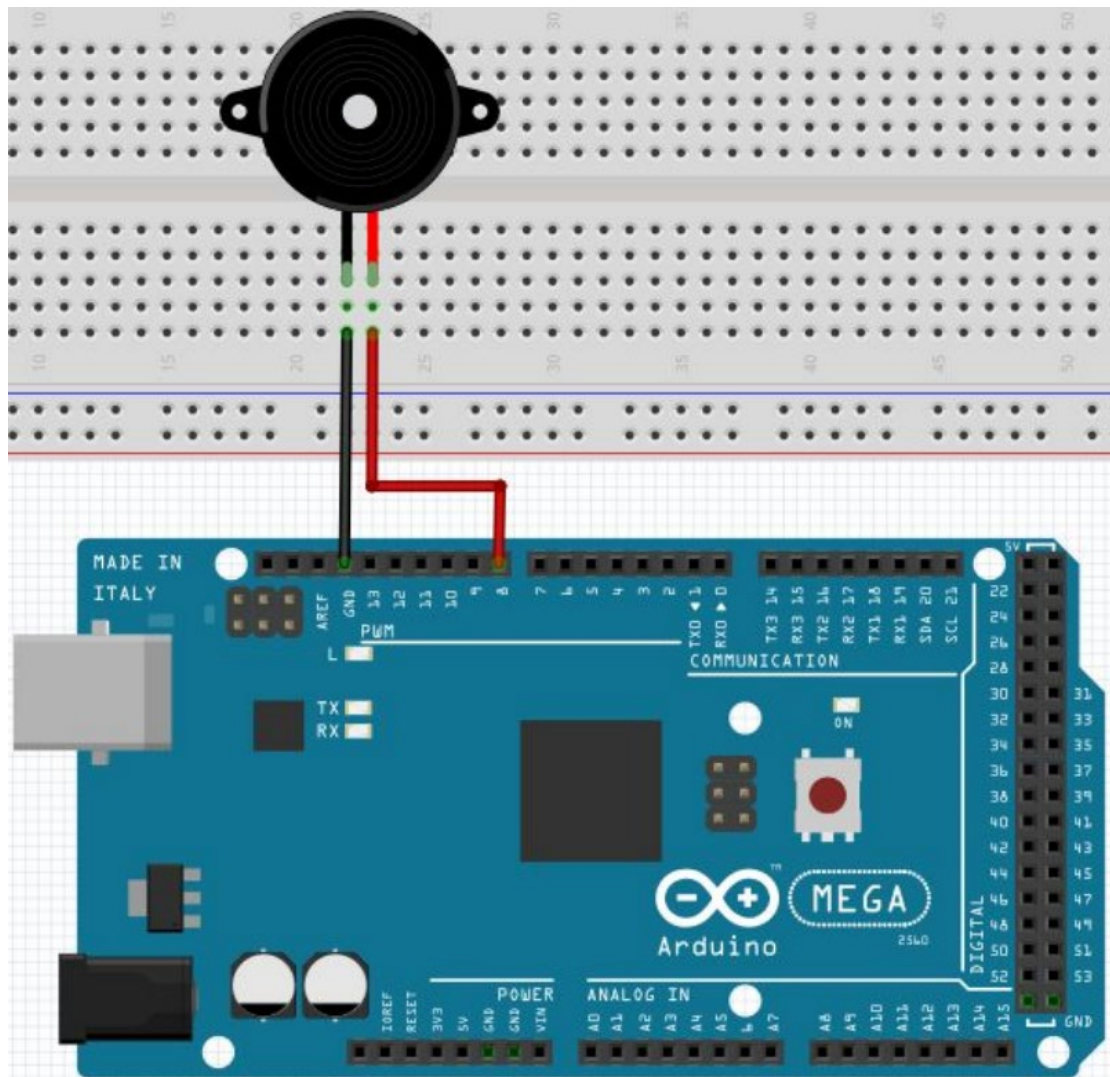
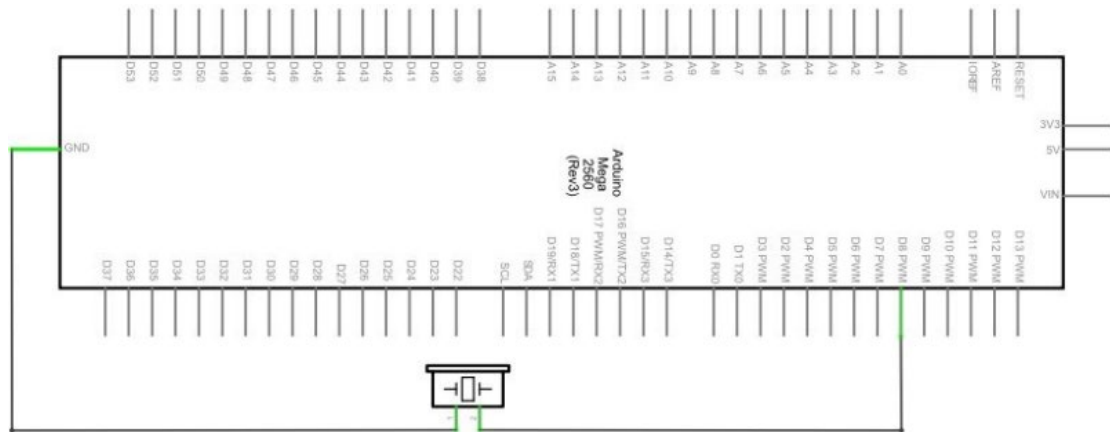
Buzzer*1

Key *1

Breadboard*1

Breadboard jumper wires

Circuit connection



When connecting the circuit, pay attention to the positive & the negative poles of the buzzer. In the photo, you can see there are red and black lines. When the circuit is finished, you can begin programming.

Sample program

Program is simple. You control the buzzer by outputting high/low level.

```
////////////////////////////////////
```

```

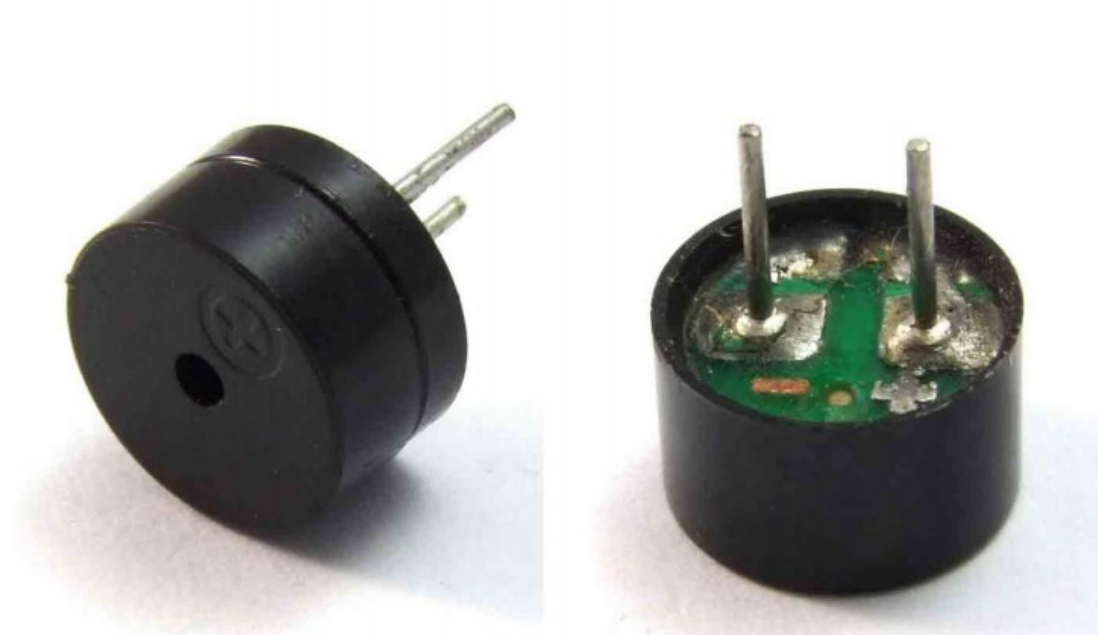
int buzzer=8;// initialize digital IO pin that controls the buzzer
void setup()
{
pinMode(buzzer,OUTPUT);// set pin mode as “output”
}
void loop()
{
digitalWrite(buzzer, HIGH); // produce sound
}
////////////////////////////////////

```

Result

After downloading the program, the buzzer experiment is completed. You can see the buzzer is ringing.

Project 9: Passive buzzer



Hardware required

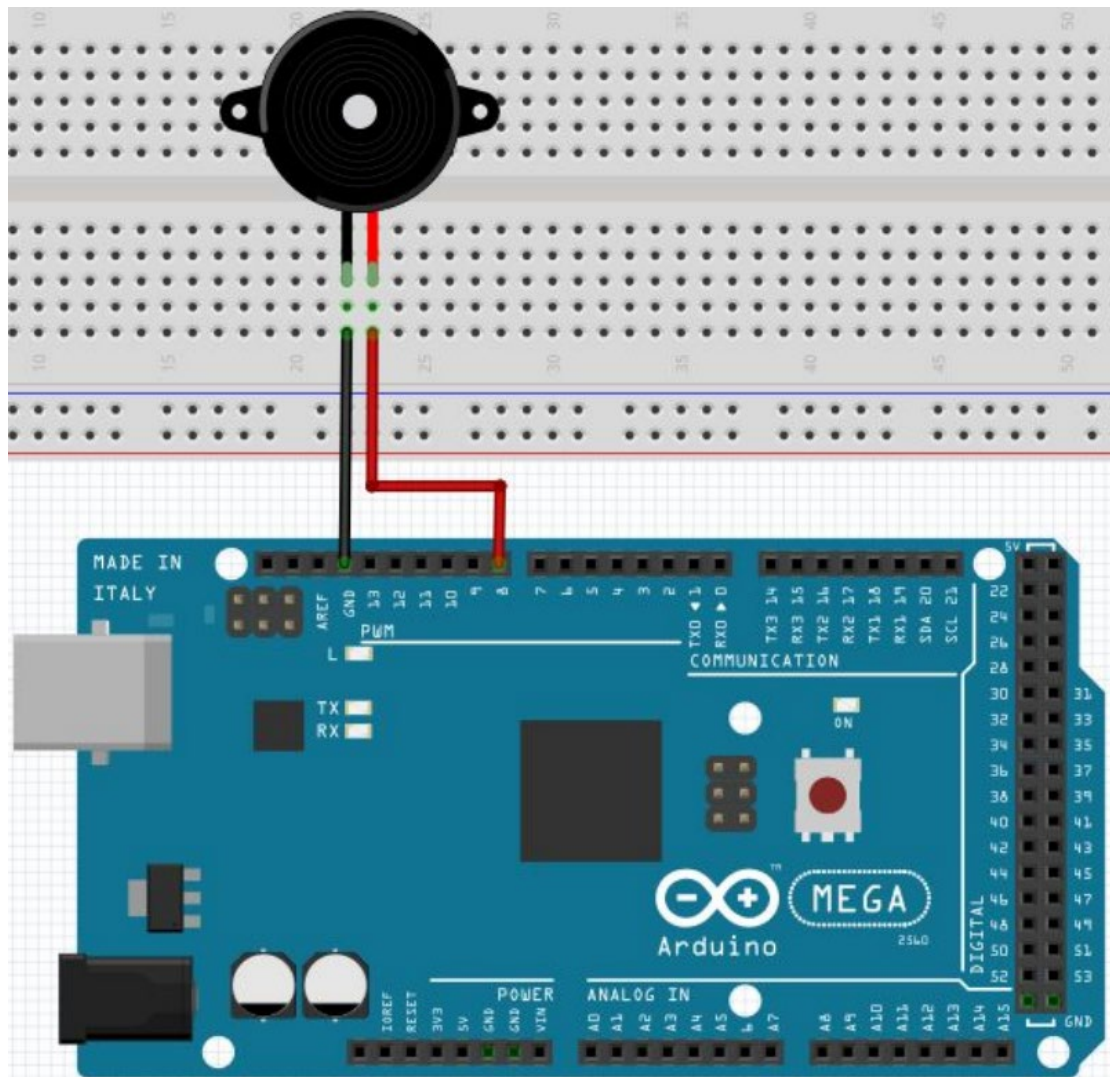
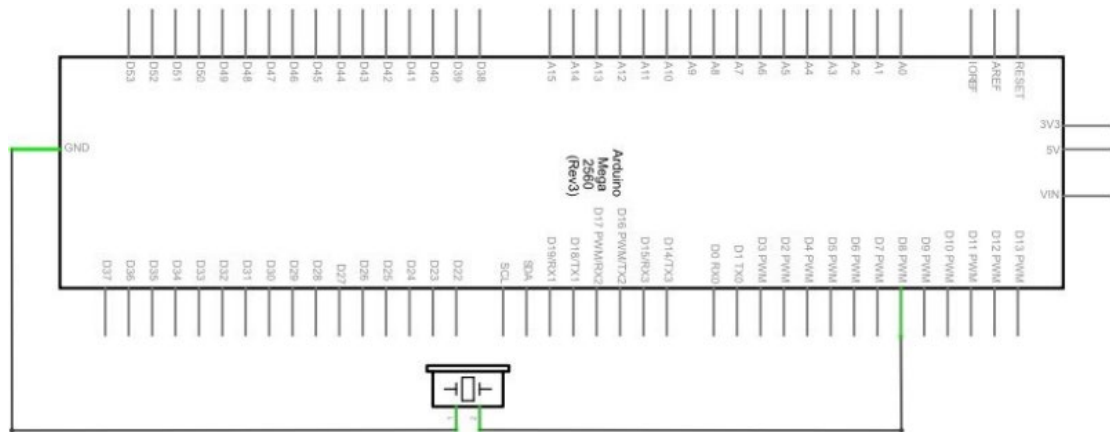
Passive buzzer*1

Key *1

Breadboard*1

Breadboard jumper wires

Circuit connection



Sample program

```

////////////////////////////////////
int buzzer=8;// select digital IO pin for the buzzer
void setup()
{
  pinMode(buzzer,OUTPUT);// set digital IO pin pattern, OUTPUT to be output

```

```

}
void loop()
{ unsigned char i,j;//define variable
while(1)
{ for(i=0;i<80;i++)// output a frequency sound
{ digitalWrite(buzzer,HIGH);// sound
delay(1);//delay1ms
digitalWrite(buzzer,LOW);//not sound
delay(1);//ms delay
}
for(i=0;i<100;i++)// output a frequency sound
{ digitalWrite(buzzer,HIGH);// sound
digitalWrite(buzzer,LOW);//not sound
delay(2);//2ms delay
}
}
}
}

```

////////////////////////////////////

After downloading the program, buzzer experiment is finished.

Project 10: Analog value reading

Introduction

In this experiment, we will begin the learning of analog I/O interfaces. On an Arduino, there are 6 analog interfaces numbered from 0 to 5. These 6 interfaces can also be used as digital ones numbered as 14-19. After a brief introduction, let's begin our project. Potentiometer used here is a typical output component of analog value that is familiar to us.

Hardware required

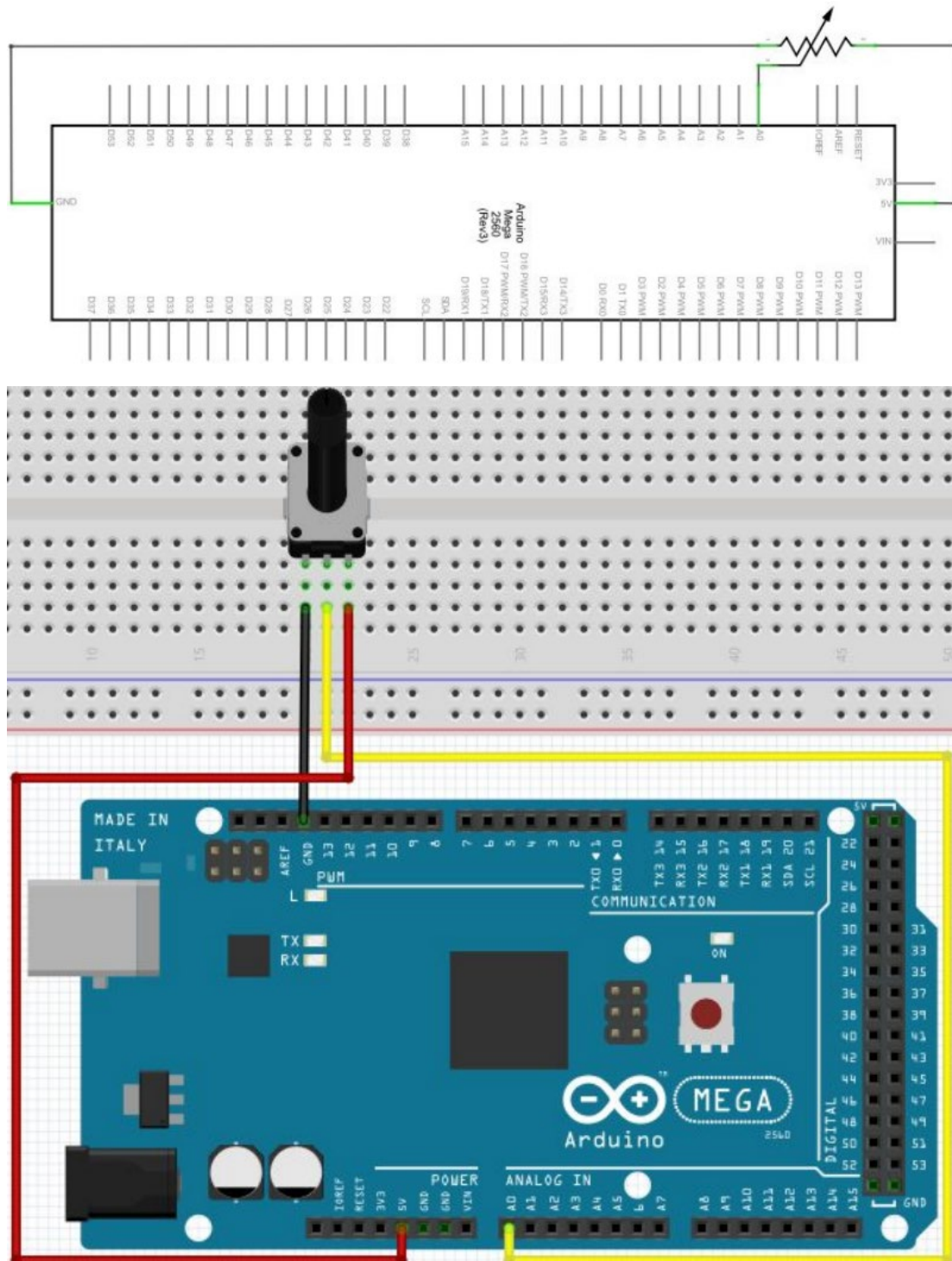
Potentiometer *1

Breadboard*1

Breadboard jumper wires * several

Circuit connection

In this experiment, we will convert the resistance value of the potentiometer to analog ones and display it on the screen. This is an application we need to master well for our future experiments. Connection circuit as below:



We use the analog interface 0.

The analog interface we use here is interface 0.

Sample program

The program compiling is simple. An `analogRead ()` Statement can read the value of the interface. The A/D acquisition of Arduino 328 is in 10 bits, so the value it reads is among 0 to 1023. One difficulty in this project is to display the value on the screen, which is actually easy to learn. First, we need to set the baud rate in `voidsetup ()`. Displaying the value is a communication between

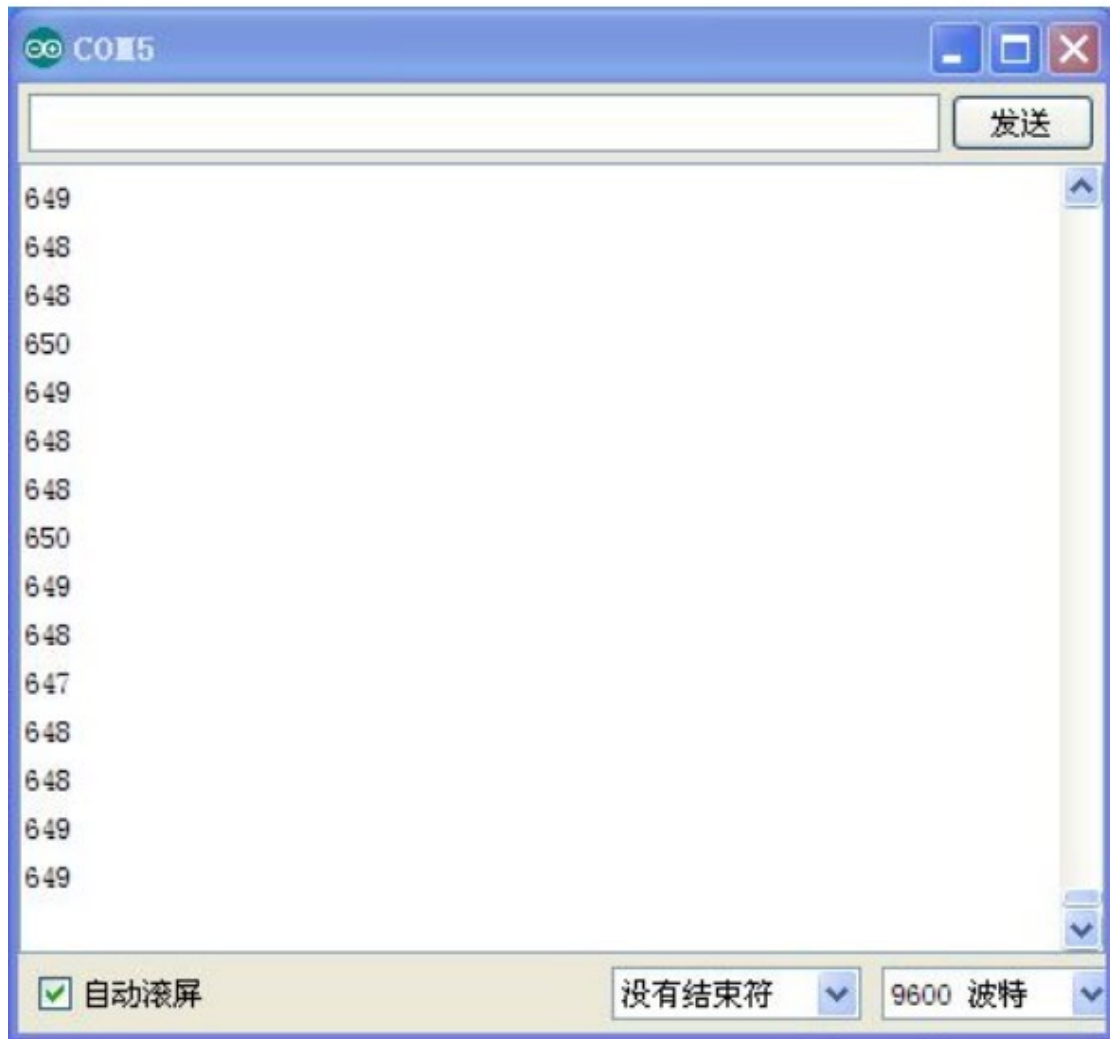
Arduino and PC, so the baud rate of the Arduino should match the the one in the PC's software set up. Otherwise, the display will be messy codes or no display at all. In the lower right corner of the Arduino software monitor window, there is a button for baud rate set up. The set up here needs to match the one in the program. The statement in the program is Serial.begin(); enclosed is the baud rate value, followed by statement for displaying. You can either use Serial.print() or Serial.println() statement.

```
////////////////////////////////////
int potpin=0;// initialize analog pin 0
int ledpin=13;// initialize digital pin 13
int val=0;// define val, assign initial value 0
void setup()
{
  pinMode(ledpin,OUTPUT);// set digital pin as "output"
  Serial.begin(9600);// set baud rate at 9600
}
void loop()
{
  digitalWrite(ledpin,HIGH);// turn on the LED on pin 13
  delay(50);// wait for 0.05 second
  digitalWrite(ledpin,LOW);// turn off the LED on pin 13
  delay(50);// wait for 0.05 second
  val=analogRead(potpin);// read the analog value of analog pin 0, and assign it to val
  Serial.println(val);// display val's value
}
////////////////////////////////////
```

Result

The sample program uses the built-in LED connected to pin 13. Each time the device reads a value, the LED blinks.

Below is the analog value it reads.



When you rotate the potentiometer knob, you can see the displayed value changes. The reading of analog value is a very common function since most sensors output analog value. After calculation, we can have the corresponding value we need.

The experiment is now completed, thank you.

Project 11: Photo resistor

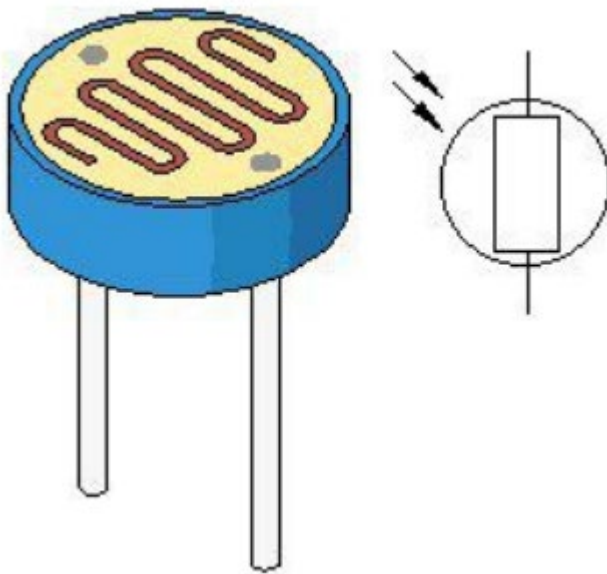


Introduction

After completing all the previous experiments, we acquired some basic understanding and knowledge about Arduino application. We have learned digital input and output, analog input and PWM. Now, we can begin the learning of sensors applications.

Photo resistor (Photovaristor) is a resistor whose resistance varies according to different incident light strength. It's made based on the photoelectric effect of semiconductor. If the incident light is intense, its resistance reduces; if the incident light is weak, the resistance increases. Photovaristor is commonly applied in the measurement of light, light control and photovoltaic conversion (convert the change of light into the change of electricity).

Photo resistor is also being widely applied to various light control circuit, such as light control and adjustment, optical switches etc.

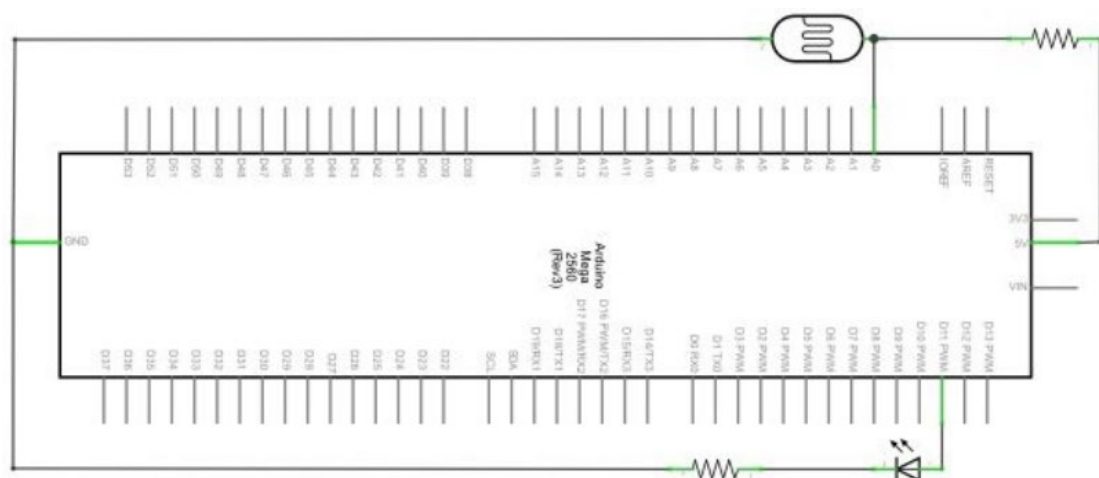


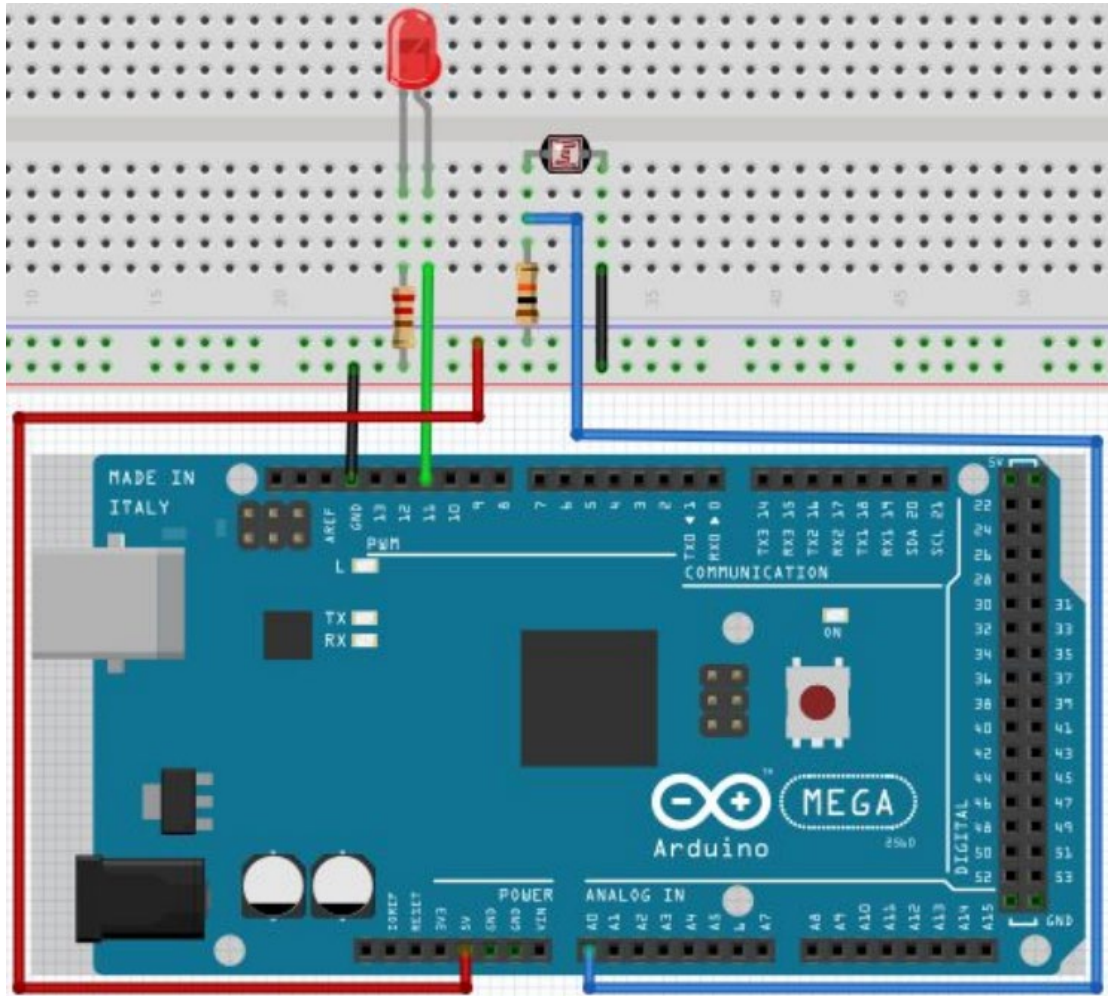
We will start with a relatively simple experiment regarding photoresistor application. Photoresistor is an element that changes its resistance as light strength changes. So we will need to read the analog values. We can refer to the PWM experiment, replacing the potentiometer with photoresistor. When there is change in light strength, there will be corresponding change on the LED.

Hardware required

Photo resistor*1
 Red M5 LED*1
 10K Ω resistor*1
 220 Ω resistor*1
 Bread board*1
 Bread board jumper wires

Circuit connection





Sample program

After the connection, let's begin the program compiling. The program is similar to the one of PWM. For change detail, please refer to the sample program below.

```
////////////////////////////////////
```

```
int potpin=0;// initialize analog pin 0, connected with photovaristor
```

```
int ledpin=11;// initialize digital pin 11, output regulating the brightness of LED
```

```
int val=0;// initialize variable va
```

```
void setup()
```

```
{
```

```
pinMode(ledpin,OUTPUT);// set digital pin 11 as "output"
```

```
Serial.begin(9600);// set baud rate at "9600"
```

```
}
```

```
void loop()
```

```
{
```

```
val=analogRead(potpin);// read the analog value of the sensor and assign it to val
```

```
Serial.println(val);// display the value of val
```

```
analogWrite(ledpin,val);// turn on the LED and set up brightness (maximum output value 255)
```

```
delay(10);// wait for 0.01
```

```
}
```

////////////////////////////////////

Result

After downloading the program, you can change the light strength around the photovaristor and see corresponding brightness change of the LED. Photovaristors has various applications in our everyday life. You can make other interesting interactive projects base on this one.

Project 12: Flame sensor

Introduction

Flame sensor (Infrared receiving triode) is specially used on robots to find the fire source. This sensor is of high sensitivity to flame. Below is a photo of it.

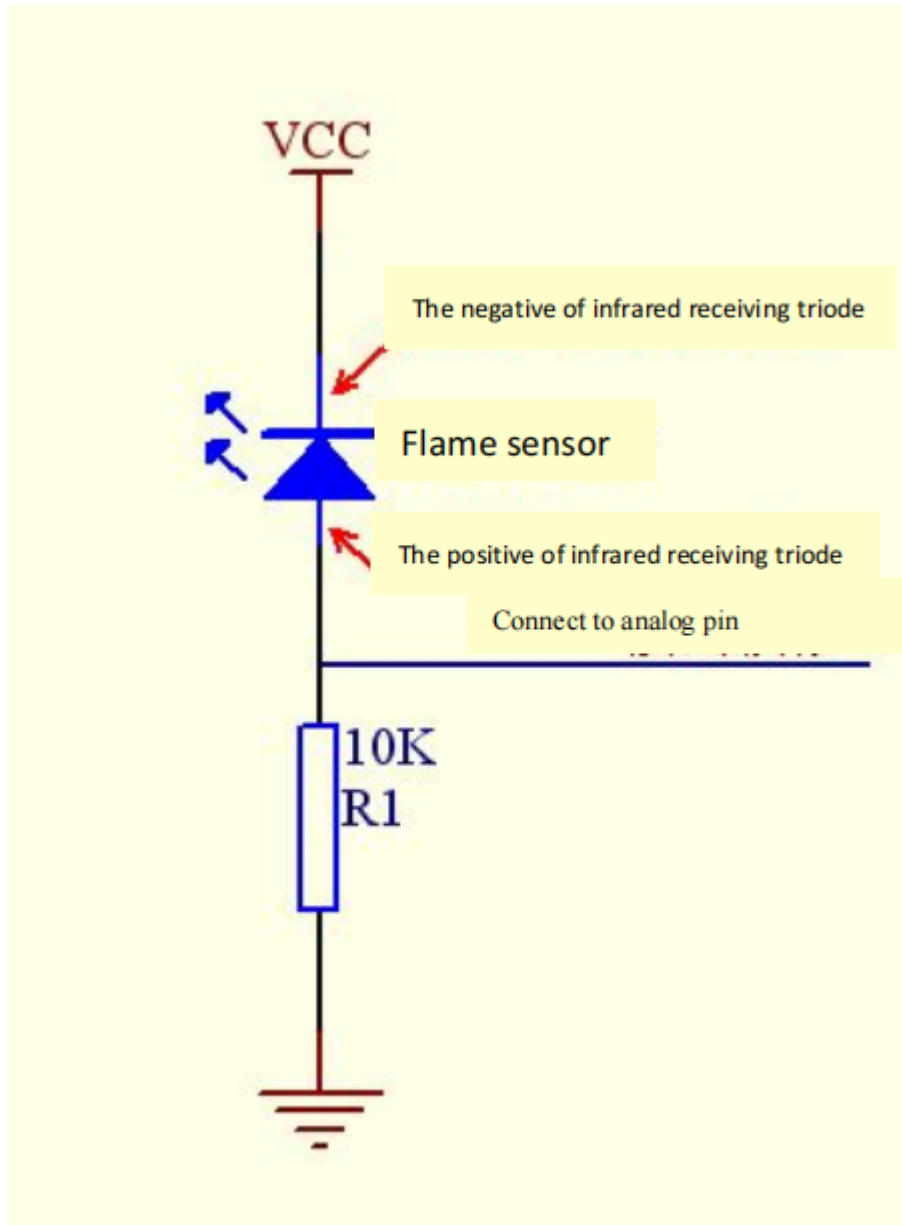


Working principle:

Flame sensor is made based on the principle that infrared ray is highly sensitive to flame. It has a specially designed infrared receiving tube to detect fire, and then convert the flame brightness to fluctuating level signal. The signals are then input into the central processor and be dealt with accordingly.

Sensor connection

The shorter lead of the receiving triode is for negative, the other one for positive. Connect negative to 5V pin, positive to resistor; connect the other end of the resistor to GND, connect one end of a jumper wire to a clip which is electrically connected to sensor positive, the other end to analog pin. As shown below:



Hardware required

Flame sensor *1

Buzzer *1

10K resistor *1

Breadboard jumper wires

Experiment connection

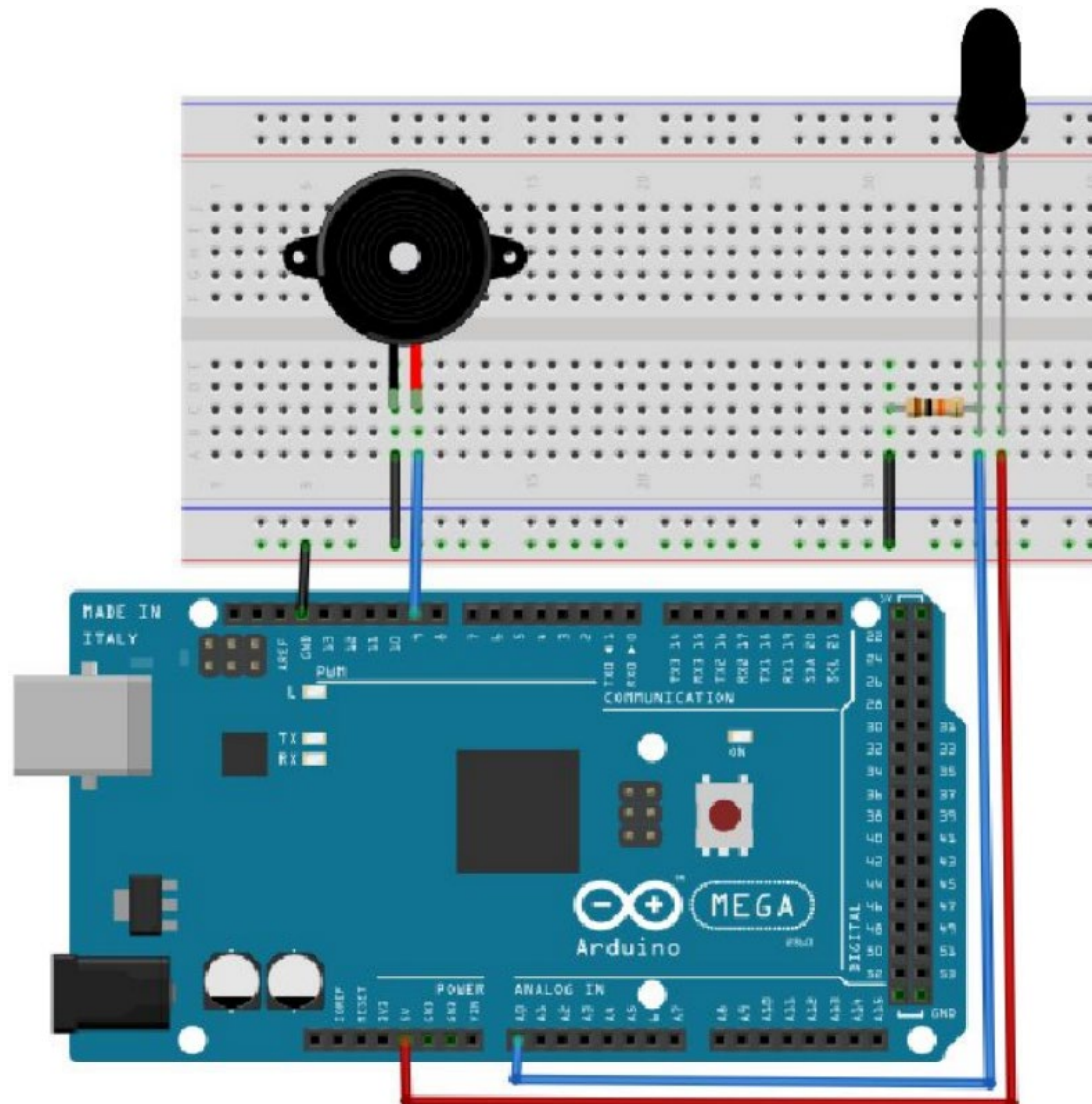
1) Connecting buzzer:

Connect the controller board, prototype board, breadboard and USB cable according to the

Arduino tutorial. Connect the buzzer to digital pin 8.

2) Connecting flame sensor:

Connect the sensor to analog pin 0.



Experiment principle

When it's approaching a fire, the voltage value the analog port reads differs. If you use a multimeter, you can know when there is no fire approaching, the voltage it reads is around 0.3V; when there is fire approaching, the voltage it reads is around 1.0V, the nearer the fire, the higher the voltage.

So in the beginning of the program, you can initialize voltage value i (no fire value); Then, continuously read the analog voltage value j and obtain difference value $k=j-i$; compare k with 0.6V (123 in binary) to determine whether or not there is a fire approaching; if yes, the buzzer will buzz.

Sample program

```
////////////////////////////////////
```

```
int flame=0;// select analog pin 0 for the sensor
```

```

int Beep=9;// select digital pin 9 for the buzzer
int val=0;// initialize variable
void setup()
{
pinMode(Beep,OUTPUT);// set LED pin as “output”
pinMode(flame,INPUT);// set buzzer pin as “input”
Serial.begin(9600);// set baud rate at “9600”
}
void loop()
{
val=analogRead(flame);// read the analog value of the sensor
Serial.println(val);// output and display the analog value
if(val>=600)// when the analog value is larger than 600, the buzzer will buzz
{
digitalWrite(Beep,HIGH);
}else
{
digitalWrite(Beep,LOW);
}
delay(500);
}
////////////////////////////////////

```

Result

This program can simulate an alarm when there is a fire. Everything is normal when there is no fire; when there is, the alarm will be set off immediately.

Project 13: Tilt switch



Introduction

Tilt switch controls the ON and OFF of an LED.

Hardware required

Ball switch*1

Led *1

220 Ω resistor*1

Breadboard jumper wires

Circuit connection

Connect the controller board, shield, breadboard and USB cable according to Arduino tutorial.

Connect the LED to digital pin 8, ball switch to analog pin 5.

Experiment principle

When one end of the switch is below horizontal position, the switch is on. The voltage of the analog port is about 5V (1023 in binary). The LED will be on. When the other end of the switch is below horizontal position, the switch is off. The voltage of the analog port is about 0V (0 in binary). The LED will be off. In the program, we determine whether the switch is on or off

according to the voltage value of the analog port, whether it's above 2.5V (512 in binary) or not.

Sample program

```
////////////////////////////////////  
void setup()  
{  
  pinMode(8,OUTPUT);// set digital pin 8 as “output”  
}  
void loop()  
{  
  int i;// define variable i  
  while(1)  
  {  
    i=analogRead(5);// read the voltage value of analog pin 5  
    if(i>512)// if larger than 512 (2.5V)  
    {  
      digitalWrite(8,LOW);// turn on LED  
    }  
    else// otherwise  
    {  
      digitalWrite(8,HIGH);// turn off LED  
    }  
  }  
}  
////////////////////////////////////
```

Result

Hold the breadboard with your hand. Tilt it to a certain extent, the LED will be on.

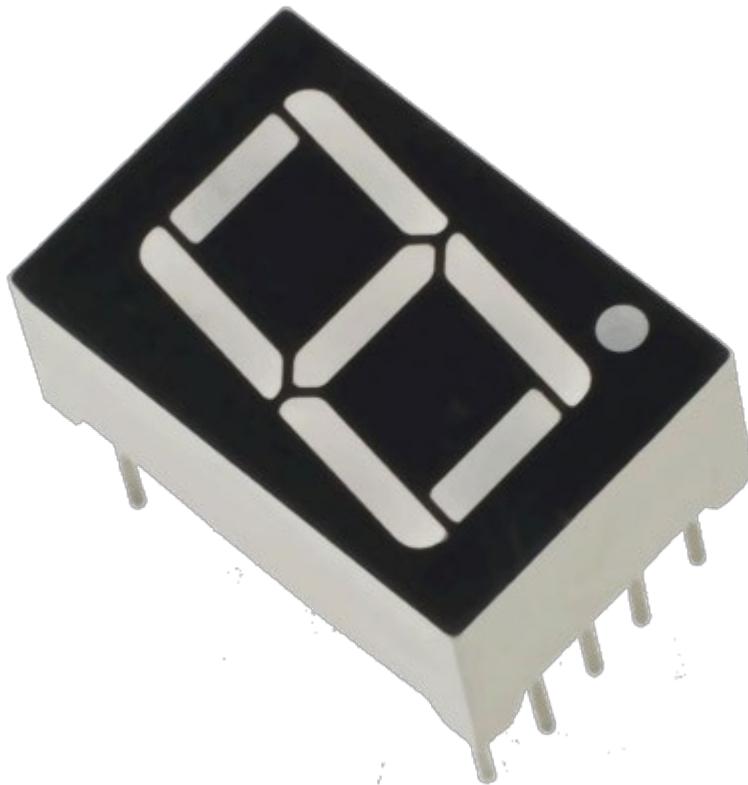
If there is no tilt, the LED will be off.

The principle of this experiment can be applied to relay control.

Experiment completed.

Thank you!

Project 14: 1-digit LED segment display



Introduction

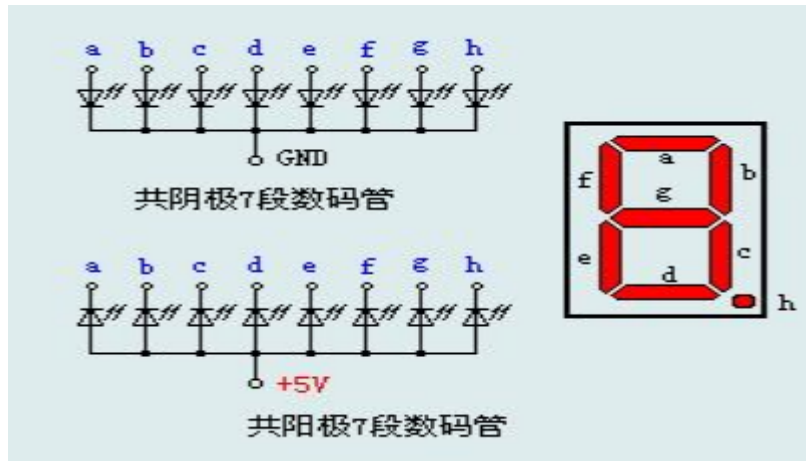
LED segment displays are common for displaying numerical information. It's widely applied on displays of electromagnetic oven, full automatic washing machine, water temperature display, electronic clock etc. It is necessary that we learn how it works.

LED segment display is a semiconductor light-emitting device. Its basic unit is a light-emitting diode (LED). LED segment display can be divided into 7-segment display and 8-segment display according to the number of segments. 8-segment display has one more LED unit (for decimal point display) than 7-segment one. In this experiment, we use a 8-segment display.

According to the wiring method of LED units, LED segment displays can be divided into display with common anode and display with common cathode. Common anode display refers to the one that combine all the anodes of LED units into one common anode (COM).

For the common anode display, connect the common anode (COM) to +5V. When the cathode level of a certain segment is low, the segment is on; when the cathode level of a certain segment is high, the segment is off. For the common cathode display, connect the common cathode (COM) to GND. When the anode level of a certain segment is high, the segment is on; when the anode level of a certain segment is low, the segment is off.

Common cathode 7-segment display



Common anode 7-segment display

Each segment of the display consists of an LED. So when you use it, you also need use a current-limiting resistor. Otherwise, LED will be burnt out. In this experiment, we use a common cathode display. As we mentioned above, for common cathode display, connect the common cathode (COM) to GND. When the anode level of a certain segment is high, the segment is on; when the anode level of a certain segment is low, the segment is off.

Hardware required

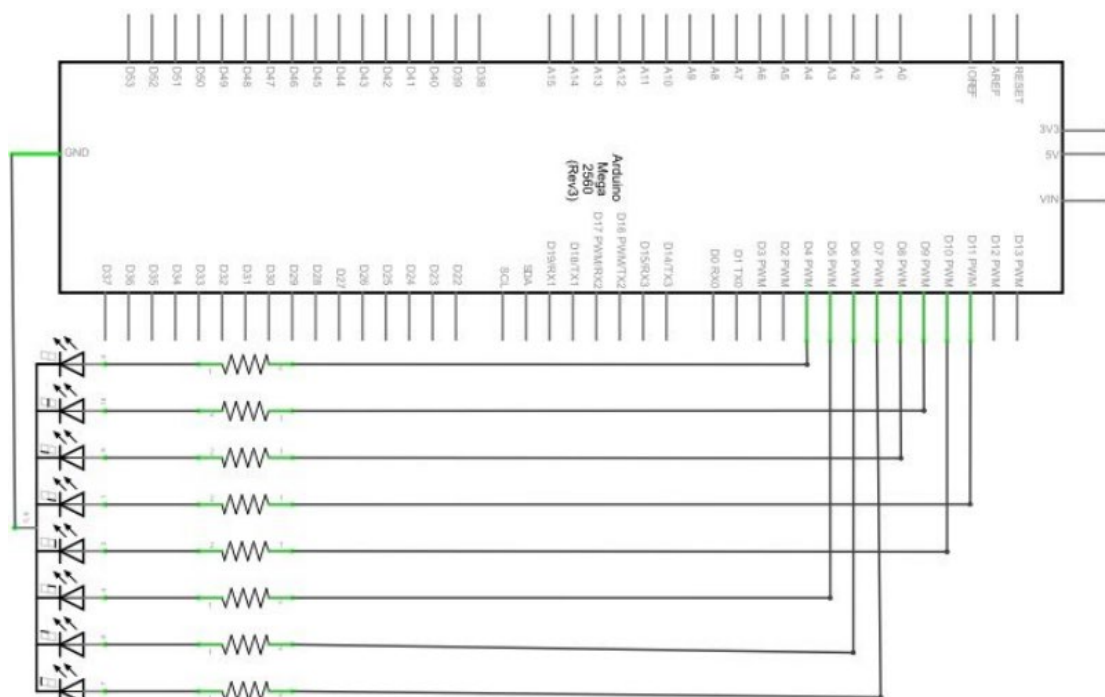
Eight-segment display*1

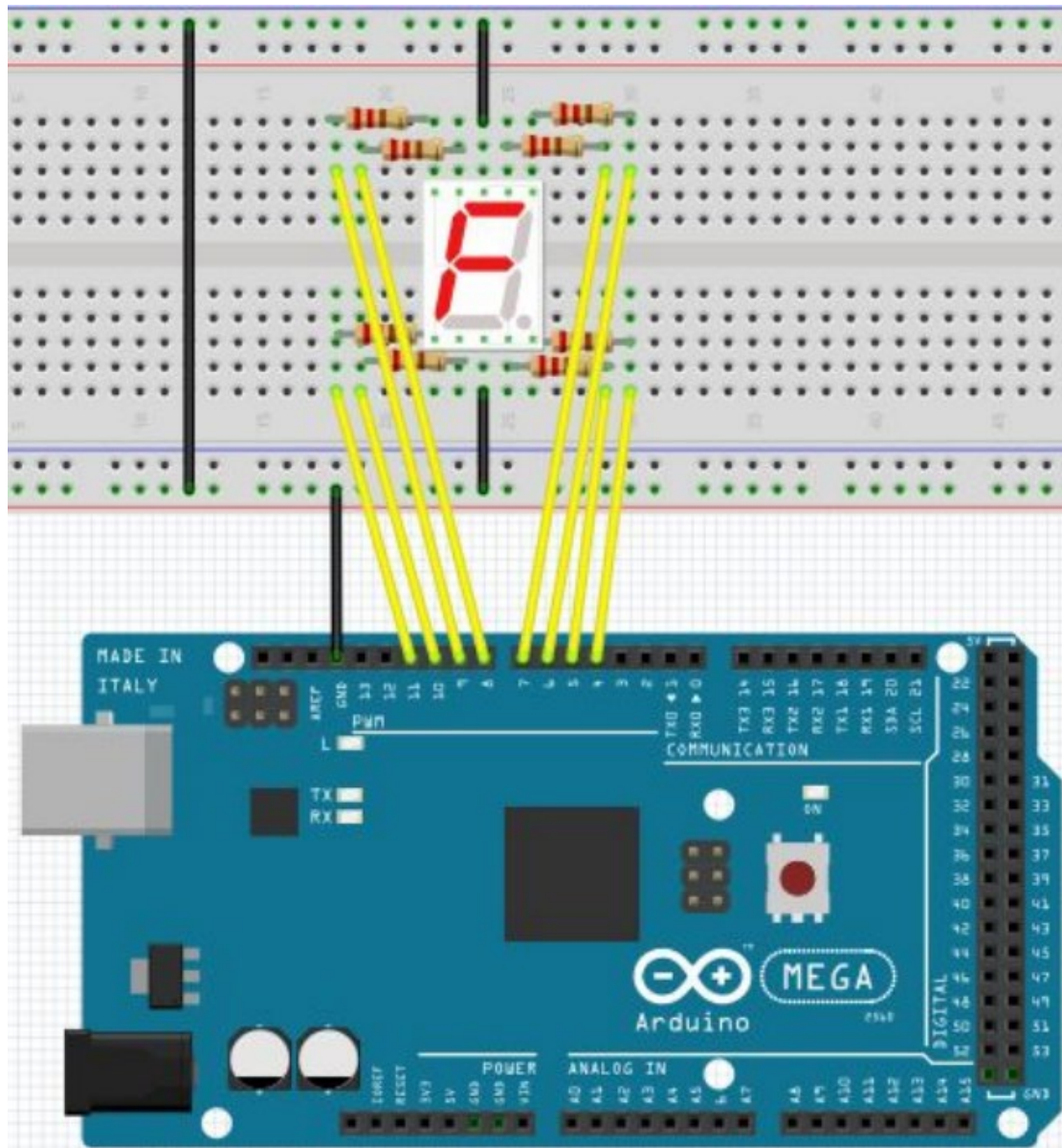
220Ω resistor*8

Breadboard*1

Breadboard jumper wires*several

Circuit connection





Sample program

There are seven segments for numerical display, one for decimal point display. Corresponding segments will be turned on when displaying certain numbers. For example, when displaying number 1, b and c segments will be turned on. We compile a subprogram for each number, and compile the main program to display one number every 2 seconds, cycling display number 0 ~ 9. The displaying time for each number is subject to the delay time, the longer the delay time, the longer the displaying time.

```
////////////////////////////////////
```

```
// set the IO pin for each segment
```

```
int a=7;// set digital pin 7 for segment a
```

```
int b=6;// set digital pin 6 for segment b
```

```
int c=5;// set digital pin 5 for segment c
```

```
int d=10;// set digital pin 10 for segment d
```

```
int e=11;// set digital pin 11 for segment e
```

```

int f=8;// set digital pin 8 for segment f
int g=9;// set digital pin 9 for segment g
int dp=4;// set digital pin 4 for segment dp
void digital_0(void) // display number 5
{
    unsigned char j;
    digitalWrite(a,HIGH);
    digitalWrite(b,HIGH);
    digitalWrite(c,HIGH);
    digitalWrite(d,HIGH);
    digitalWrite(e,HIGH);
    digitalWrite(f,HIGH);
    digitalWrite(g,LOW);
    digitalWrite(dp,LOW);
}
void digital_1(void) // display number 1
{
    unsigned char j;
    digitalWrite(c,HIGH);// set level as “high” for pin 5, turn on segment c
    digitalWrite(b,HIGH);// turn on segment b
    for(j=7;j<=11;j++)// turn off other segments
        digitalWrite(j,LOW);
    digitalWrite(dp,LOW);// turn off segment dp
}
void digital_2(void) // display number 2
{
    unsigned char j;
    digitalWrite(b,HIGH);
    digitalWrite(a,HIGH);
    for(j=9;j<=11;j++)
        digitalWrite(j,HIGH);
    digitalWrite(dp,LOW);
    digitalWrite(c,LOW);
    digitalWrite(f,LOW);
}
void digital_3(void) // display number 3
{
    digitalWrite(g,HIGH);
    digitalWrite(a,HIGH);
    digitalWrite(b,HIGH);
    digitalWrite(c,HIGH);
    digitalWrite(d,HIGH);
    digitalWrite(dp,LOW);
    digitalWrite(f,LOW);
    digitalWrite(e,LOW);
}

```

```

}
void digital_4(void) // display number 4
{digitalWrite(c,HIGH);
digitalWrite(b,HIGH);
digitalWrite(f,HIGH);
digitalWrite(g,HIGH);
digitalWrite(dp,LOW);
digitalWrite(a,LOW);
digitalWrite(e,LOW);
digitalWrite(d,LOW);
}
void digital_5(void) // display number 5
{
unsigned char j;
digitalWrite(a,HIGH);
digitalWrite(b, LOW);
digitalWrite(c,HIGH);
digitalWrite(d,HIGH);
digitalWrite(e, LOW);
digitalWrite(f,HIGH);
digitalWrite(g,HIGH);
digitalWrite(dp,LOW);
}
void digital_6(void) // display number 6
{
unsigned char j;
for(j=7;j<=11;j++)
digitalWrite(j,HIGH);
digitalWrite(c,HIGH);
digitalWrite(dp,LOW);
digitalWrite(b,LOW);
}
void digital_7(void) // display number 7
{
unsigned char j;
for(j=5;j<=7;j++)
digitalWrite(j,HIGH);
digitalWrite(dp,LOW);
for(j=8;j<=11;j++)
digitalWrite(j,LOW);
}
void digital_8(void) // display number 8
{
unsigned char j;

```

```

for(j=5;j<=11;j++)
digitalWrite(j,HIGH);
digitalWrite(dp,LOW);
}
void digital_9(void) // display number 5
{
unsigned char j;
digitalWrite(a,HIGH);
digitalWrite(b,HIGH);
digitalWrite(c,HIGH);
digitalWrite(d,HIGH);
digitalWrite(e, LOW);
digitalWrite(f,HIGH);
digitalWrite(g,HIGH);
digitalWrite(dp,LOW);
}
void setup()
{
int i;// set variable
for(i=4;i<=11;i++)
pinMode(i,OUTPUT);// set pin 4-11 as “output”
}
void loop()
{
while(1)
{
digital_0();// display number 0
delay(1000);// wait for 1s
digital_1();// display number 1
delay(1000);// wait for 1s
digital_2();// display number 2
delay(1000); // wait for 1s
digital_3();// display number 3
delay(1000); // wait for 1s
digital_4();// display number 4
delay(1000); // wait for 1s
digital_5();// display number 5
delay(1000); // wait for 1s
digital_6();// display number 6
delay(1000); // wait for 1s
digital_7();// display number 7
delay(1000); // wait for 1s
digital_8();// display number 8
delay(1000); // wait for 1s

```

```
digital_9();// display number 9
delay(1000); // wait for 1s
}
}
////////////////////////////////////
```

Result

LED segment display displays number 0 to 9.
